DESIGN AND EVALUATION OF A HYBRID PROMPT-AWARE, MULTI-OBJECTIVE LARGE LANGUAGE MODEL ROUTING SYSTEM WITH AUTOMATIC FALLBACK

BY

TERSEER SOLOMON TERZUNGWE-TEMBE ([STUDENT ID])

SODIQ ABDULWARIS ([STUDENT ID])

[FULL NAME: MARYAM] ([STUDENT ID])

[FULL NAME: WAHIDA] ([STUDENT ID])

[FULL NAME: KABIRU] ([STUDENT ID])

[FULL NAME: ADRIEL] ([STUDENT ID])

A PROJECT SUBMITTED TO
THE DEPARTMENT OF SOFTWARE ENGINEERING, FACULTY OF COMPUTING, NILE UNIVERSITY OF NIGERIA,
IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE AWARD OF THE DEGREE OF BACHELOR OF SCIENCE (B.Sc. HONS) IN SOFTWARE ENGINEERING,
NILE UNIVERSITY OF NIGERIA

[MONTH] 2026

NILE UNIVERSITY OF NIGERIA

STATUS CONFIRMATION FOR BACHELOR'S PROJECT

PROJECT TITLE: DESIGN AND EVALUATION OF A HYBRID PROMPT-AWARE, MULTI-OBJECTIVE LARGE LANGUAGE MODEL ROUTING SYSTEM WITH AUTOMATIC FALLBACK

ACADEMIC SESSION: 2026/2027

We, TERSEER SOLOMON TERZUNGWE-TEMBE, SODIQ ABDULWARIS, [FULL NAME: MARYAM], [FULL NAME: WAHIDA], [FULL NAME: KABIRU], [FULL NAME: ADRIEL], agree to allow this degree project to be kept at the Library under the following terms:

1. This Degree Project is the property of Nile University of Nigeria.
2. The library has the right to make copies for educational purposes only.
3. The library is allowed to make copies of this report for educational exchange between higher educational institutions.
4. ** Please Mark (√)

☐	CONFIDENTIAL	(Contains information of high security or of great importance to Nigeria as stipulated under the OFFICIAL SECRETS ACT)
☐	RESTRICTED	(Contains restricted information as determined by the Organization/institution where research was
☐	FREE ACCESS	

Approved by,

_____ _____

(STUDENTS' SIGNATURES) (SUPERVISOR'S SIGNATURE)

Permanent Address :

[Permanent address of the group leader]

Date: _____

Supervisor's name:

[Supervisor's full name]

Date: _____

CERTIFICATION

This is to certify that the project titled "Design and Evaluation of a Hybrid Prompt-Aware, Multi-Objective Large Language Model Routing System with Automatic Fallback" by TERSEER SOLOMON TERZUNGWE-TEMBE ([STUDENT ID]), SODIQ ABDULWARIS ([STUDENT ID]), [FULL NAME: MARYAM] ([STUDENT ID]), [FULL NAME: WAHIDA] ([STUDENT ID]), [FULL NAME: KABIRU] ([STUDENT ID]) and [FULL NAME: ADRIEL] ([STUDENT ID]) has been read and approved as meeting the requirements for the award of the degree of Bachelor of Science (B.Sc. Hons) in Software Engineering, Department of Software Engineering, Faculty of Computing, Nile University of Nigeria, Abuja.

…….

[Supervisor's full name]

PROJECT SUPERVISOR

……

Assoc. Prof. Ibrahim Salihu Anka

HOD SOFTWARE ENGINEERING

…….

Prof. Joshua Abah

DEAN FACULTY OF COMPUTING

.....

EXTERNAL EXAMINER

DECLARATION

We hereby declare that this project report is our own work and has not been submitted, in whole or in part, for any other degree or qualification. All sources consulted have been duly cited and acknowledged in the references, and any use of generative AI tools has been declared in this report.

Name (Student ID): TERSEER SOLOMON TERZUNGWE-TEMBE ([STUDENT ID])
Signature: ……

Name (Student ID): SODIQ ABDULWARIS ([STUDENT ID]) Signature:
……

Name (Student ID): [FULL NAME: MARYAM] ([STUDENT ID]) Signature:
……

Name (Student ID): [FULL NAME: WAHIDA] ([STUDENT ID]) Signature:
……

Name (Student ID): [FULL NAME: KABIRU] ([STUDENT ID]) Signature:
……

Name (Student ID): [FULL NAME: ADRIEL] ([STUDENT ID]) Signature:
……

Supervisor : …….
[SUPERVISOR'S FULL NAME]

STATEMENT OF INDIVIDUAL CONTRIBUTIONS

Each member's planned role and main contributions are listed below. These follow the team's task distribution; each member will confirm or correct their own entry, in their own words, before signing.

S/N	Full name	Student ID	Role	Main contributions (modules, chapters, tasks)	Signature
1	Terseer Solomon Terzungwe-Tembe	[STUDENT ID]	Group leader; routing algorithm and machine learning developer	Suitability predictor and selection policy (3.6.5, 3.7); sprint coordination; supervisor liaison	
2	Sodiq Abdulwaris	[STUDENT ID]	System architect; model integration, API and database developer	Architecture (3.6.1); provider adapters and reliability controller; database design (3.6.4)	
3	[FULL NAME: MARYAM]	[STUDENT ID]	Backend developer; testing and evaluation developer	Gateway backend; replay and fault-injection harness; evaluation plan (3.9)	
4	[FULL NAME: WAHIDA]	[STUDENT ID]	System architect; documentation lead; testing and evaluation	Requirements (3.2, 3.5); Chapter Two coordination and references; test plan	
5	[FULL NAME: KABIRU]	[STUDENT ID]	Frontend and backend developer; documentation	Administrator dashboard (3.6.6); requirements extraction; report assembly	
6	[FULL NAME: ADRIEL]	[STUDENT ID]	[Role to be agreed]	[To be agreed]	

DECLARATION ON THE USE OF GENERATIVE AI TOOLS

Generative AI tools were used in preparing this proposal, as declared in the table below. The team remains responsible for all content and has checked, or will check before submission, every AI-assisted section against the original sources.

Tool and version	Task it was used for	Chapter / section	How the output was verified or edited
Claude (Anthropic), Claude Opus 5.5 in Claude Code, November 2026	Drafted the proposal text from the team's research notes, design decisions and literature records; drew Figures 2.1 and 3.1 to 3.6 with Python scripts; formatted the document in the department template	Front matter; Chapters One to Three	[To be completed by the team: each section read and edited by its owner; every citation, number and figure checked against the original source]
AI-assisted literature summaries (earlier sessions; tool and version to be confirmed)	Reading notes for the 90 studies in the review register	Chapter Two	Summaries not treated as reading; key studies to be read in full and every reported figure checked

DEDICATION

We dedicate this project to [to be written by the team].

ACKNOWLEDGEMENT

We thank our supervisor, [Supervisor's full name], for guidance throughout this proposal, and the staff of the Department of Software Engineering, Nile University of Nigeria, for the Research Methodology modules that shaped its structure. We also thank our families for their support.

ABSTRACT

Applications that use large language models usually send every request to a single model. This wastes money on requests a cheaper model could answer, and choosing a weaker model instead loses quality on hard requests. Research routers predict which model should answer each request, but most choose between two models, optimise only average cost and quality, and are rarely compared with simple alternatives under the same conditions. Commercial gateways offer failover but publish no evaluation of their routing decisions. This project aims to design, implement and evaluate a hybrid prompt-aware routing system that selects, for each request, a model from a mixed pool of local and hosted models under quality, cost and latency constraints, with automatic fallback when a provider fails. The study follows Design Science Research, with development in six two-week Scrum sprints. The router will be built in Python with FastAPI behind one OpenAI-compatible endpoint. It will predict each model's quality on a prompt by combining a lightweight task classifier with embedding-based retrieval of scored past prompts, then select the cheapest model whose predicted quality meets a threshold within the request's cost cap and deadline. A reliability controller with circuit breakers and bounded retries will handle provider failures. The router will be trained and evaluated offline on stored, scored outputs from a public routing benchmark, timed on a small capped live pilot, and tested under injected failures. It will be compared with fixed-model, cheapest-eligible, rule-based and single-component baselines using paired statistical tests and ablations. Expected outputs are an open-source routing gateway, a reusable evaluation harness and an evidence-based answer to whether the hybrid approach lowers cost at acceptable quality compared with simpler routing. A finding that simpler methods perform as well will also be reported, since it would guide developers choosing a routing method.
Keywords: circuit breaker; cost-aware routing; design science research; large language models;

model selection

TABLE OF CONTENTS

CERTIFICATION i
DECLARATION iii
STATEMENT OF INDIVIDUAL CONTRIBUTIONS iv
DECLARATION ON THE USE OF GENERATIVE AI TOOLS v
DEDICATION vi
ACKNOWLEDGEMENT vii
ABSTRACT viii
LIST OF TABLES xii
LIST OF FIGURES xiii
LIST OF ABBREVIATIONS xv
CHAPTER ONE 1
INTRODUCTION 1
1.1 Background of the Study 1
1.2 Statement of the Problem 2
1.3 Aim and Objectives 3
1.4 Research Questions 4
1.5 Methodology Overview 5
1.6 Significance of the Study 5
1.7 Scope of the Study 6
1.8 Limitations of the Study 6
1.9 Contributions of the Project 7
1.10 Organisation of the Report 7
1.11 Definition of Operational Terms 8
CHAPTER TWO 9
LITERATURE REVIEW 9
2.1 Review Methodology 9
2.2 Conceptual and Theoretical Framework 10
2.3 Enabling Technologies and Techniques 11
2.4 Review of Related Works 13
2.4.1 Learned pre-generation routing between models 13
2.4.2 Embedding retrieval and historical-performance features 14
2.4.3 Cascades and joint quality–cost–latency optimisation 15

2.4.4 Latency targets and deadline-aware selection 16
2.4.5 Failure handling and fallback 16
2.4.6 Evaluating routers 17
2.5 Review of Existing Systems 18
2.6 Summary of Reviewed Works 19
2.7 Research Gap and Justification of the Study 22
2.8 Chapter Summary 23
CHAPTER THREE 24
METHODOLOGY, SYSTEM ANALYSIS AND DESIGN 24
3.1 Research and Development Methodology 24
3.2 Requirements Elicitation 25
3.3 Analysis of the Existing System 26
3.3.1 Description of the Existing System 26
3.3.2 Limitations of the Existing System 26
3.4 The Proposed System 26
3.4.1 Description of the Proposed System 26
3.4.2 Justification for the Proposed System 27
3.5 Requirements Specification 27
3.5.1 Functional Requirements 28
3.5.2 Non-Functional Requirements 29
3.5.3 Use Case Model 30
3.6 System Design 31
3.6.1 System Architecture 31
3.6.2 Structural Design (Class Diagram) 32
3.6.3 Behavioural Design (Sequence and Activity Diagrams) 32
3.6.4 Database Design 33
3.6.5 Algorithm and Model Design 35
3.6.6 User Interface Design 37
3.6.7 Security and Privacy Design 38
3.6.8 Requirements Traceability 38
3.7 Data Collection and Preparation 39
3.8 Tools and Technologies 40
3.9 Evaluation Plan 41

3.10 Ethical Considerations 42
3.11 Risk Assessment 42
3.12 Work Plan 43
3.13 Budget 44
3.14 Expected Outcomes 44
3.15 Chapter Summary 44
REFERENCES 46
APPENDICES 54

LIST OF TABLES

Table 2.1: Literature search strategy
Table 2.2: Comparison of suitability prediction techniques
Table 2.3: Feature comparison of existing systems and the proposed system
Table 2.4: Summary of related works
Table 2.5: Mapping of research gaps to project objectives
Table 3.1: Limitations of current practice and the proposed response
Table 3.2: Functional requirements
Table 3.3: Non-functional requirements
Table 3.4: Use case description for "Send chat request"
Table 3.5: Main relational tables (draft)
Table 3.6: Draft traceability from requirements to design and tests
Table 3.7: Tools and technologies
Table 3.8: Evaluation plan by objective
Table 3.9: Risk assessment (L = likelihood, I = impact)

Table 3.10: Sprint plan
Table 3.11: Budget

LIST OF FIGURES

Figure 2.1: Conceptual framework of the study
Figure 3.1: Design Science Research activities applied to the project
Figure 3.2: Use case diagram
Figure 3.3: System architecture and deployment
Figure 3.4: Entity relationship diagram
Figure 3.5: Routing and fallback algorithm
Figure 3.6: Wireframe of the administrator dashboard overview (sample data)

LIST OF ABBREVIATIONS

Each abbreviation is also spelt out in full at its first use in the text.

Abbreviation	Meaning
API	Application Programming Interface
AUROC	Area Under the Receiver Operating Characteristic curve
DSR	Design Science Research
ECE	Expected Calibration Error
ERD	Entity Relationship Diagram
FR	Functional Requirement
GPU	Graphics Processing Unit
HTTP	Hypertext Transfer Protocol
k-NN	k-Nearest Neighbours
LLM	Large Language Model
MoSCoW	Must, Should, Could, Won't
NDPA	Nigeria Data Protection Act
NFR	Non-Functional Requirement
p95	95th percentile
RQ	Research Question
SLO	Service-Level Objective
TLS	Transport Layer Security
TTFT	Time To First Token
UML	Unified Modeling Language
VRAM	Video Random-Access Memory

CHAPTER ONE

INTRODUCTION

This chapter introduces the study. It sets out the background to large language model routing, states the problem the project will address, and presents the aim, objectives and research questions. It then summarises the methodology, the significance, scope and limitations of the study, the expected contributions, the organisation of the report and the operational terms used throughout.

1.1 Background of the Study

Large language models (LLMs) are now a standard component of software products. Developers call them through hosted application programming interfaces (APIs) or run open-weight models on their own hardware, and a single application often has access to several models that differ in quality, price, speed and reliability. The models do not differ uniformly. A small model can answer a simple factual or formatting request as well as a frontier model, while a multi-step reasoning or coding task may need the larger model. Benchmark studies show this spread clearly: when stored responses from many models are compared prompt by prompt, an oracle that always picks the cheapest model able to answer correctly scores far above any single model, at a fraction of the cost (Hu et al., 2024; Li et al., 2026).

This spread is the motivation for LLM routing: choosing, for each request, which model should answer it. Research on routing has grown quickly since 2023. Learned routers predict, before any model is called, whether a cheaper model is likely to be good enough (Ding et al., 2024; Ong et al., 2025). Cascades call a cheap model first and escalate when its answer looks weak (Chen et al., 2024a). Later work treats routing as an optimisation problem over predicted quality and predicted cost (Dekoninck et al., 2025) and reports that simple nearest-neighbour methods built on embeddings of past prompts can match more complex learned routers (Varangot-Reille et al., 2026a). Two recent surveys describe routing as a resource-optimisation problem and note that evaluations across papers are rarely comparable (Moslem & Kelleher, 2026; Varangot-Reille et al., 2026b).

Routing has also moved into industry. Gateways such as OpenRouter, LiteLLM and Portkey place one API in front of many providers and offer fallbacks when a provider fails (LiteLLM, n.d.-a; OpenRouter, n.d.-a; Portkey, n.d.-b). Some now select a model per prompt, for

example by sorting requests into complexity tiers (Dholakia, 2026; OpenRouter, n.d.-c). Their documentation describes what the products do, but it does not publish evaluations of how well the per-prompt choice performs against simpler alternatives.

Operational reliability is a separate concern. Hosted LLM services fail in ways that ordinary web services also fail, through timeouts, rate limits and server errors, and in some ways that are harder to see. A study of production incidents in generative AI cloud services found that about half involved degraded rather than failed service and that many were first detected by people rather than by monitors (Yan et al., 2025). Gateway failures can also be silent: a response can arrive with a success status and still be invalid (Pandey & Singh, 2026b). Retries can make things worse, since retry traffic sustained more than half of the metastable failures in one survey of public incidents (Huang et al., 2022).

These issues have a local dimension for Nigerian developers and student teams. The hosted models considered for this project are priced per token in United States dollars, so a team paying in naira carries both the per-token cost and exchange-rate exposure on every request sent to an expensive model. Many small teams also own consumer graphics hardware, such as the two RTX 4070 laptops available to this project team, which can run a small open-weight model locally at no marginal API cost. A router that sends easy requests to a local or cheap model and reserves the expensive model for requests that need it is therefore of practical interest, provided its quality, cost and latency are measured.

1.2 Statement of the Problem

What is. Most applications that use LLMs send every request to one model fixed in code. Sending everything to a frontier model pays its price on requests a smaller model could answer; sending everything to a small model loses quality on hard requests (Ding et al., 2024; Ong et al., 2025). Research routers address this choice, but most route between two models, optimise quality and cost offline, and do not handle failures (Chen et al., 2024b; Ding et al., 2024; Ong et al., 2025). When routers are compared under shared conditions, many do not beat the best single model and all remain far from the oracle (Hu et al., 2024; Li et al., 2026). Commercial gateways handle failover and some route per prompt, but their per-prompt decisions are not published or evaluated against baselines (Dholakia, 2026; OpenRouter, n.d.-c).

What ought to be. A developer with several models available should be able to send a request with optional cost and time limits and have it answered by the cheapest model expected to meet the quality need, within those limits, with bounded recovery when a provider fails. The claim that such a router saves money without losing quality should be supported by measurements against simple baselines, with each part of the router tested for its contribution.

The gap and its consequences. The literature reviewed in Chapter Two contains strong components (learned suitability predictors, embedding retrieval of past outcomes, constrained cost–quality selection, and resilience patterns for remote calls), but they are usually evaluated in isolation. Whether combining lightweight classification with embedding-derived suitability features and constrained selection actually improves on simpler routing methods has not been established, and recent benchmarks suggest that added complexity often does not help (Li et al., 2026; Varangot-Reille et al., 2026a). Without that evidence, teams either keep paying for the largest model or adopt routing features whose benefit is unknown.

This project investigates whether combining lightweight classification and embedding-derived suitability features with constrained cost–quality–latency selection improves on simpler routing baselines. Operational fallback is evaluated separately for resilience. This project therefore seeks to design, build and evaluate such a router and to report honestly whether the hybrid approach is worth its complexity.

1.3 Aim and Objectives

Aim: To design, implement and evaluate a hybrid prompt-aware routing system that selects, for each request, a large language model from a mixed pool of local and hosted models under quality, cost and latency constraints, with automatic operational fallback.

Objectives:

1. To specify the functional and non-functional requirements of the routing system from an analysis of existing gateways, router research and provider documentation, by the end of Sprint 1.
2. To design the architecture, data model and core algorithms of the system (gateway, constraint filter, suitability predictor, selection policy and reliability controller) using UML models, by the end of Sprint 2.

3. To implement a model-specific suitability predictor that combines a lightweight task classifier with embedding-based retrieval of scored past prompts, trained on licensed benchmark outcome data, by the end of Sprint 4.
4. To implement a constrained selection policy behind one OpenAI-compatible endpoint that routes to three or four local and hosted models, by the end of Sprint 4.
5. To implement and evaluate a reliability controller with health tracking, circuit breakers and bounded fallback, using injected provider failures, by the end of Sprint 5.
6. To evaluate the hybrid router against fixed-model, cheapest-eligible, rule-based and single-component baselines on quality, cost, latency and constraint violations, including an ablation of each predictor component, by the end of Sprint 6.

1.4 Research Questions

The study will answer one research question for each objective.

- RQ1: What functional and non-functional requirements must a prompt-aware router meet to serve developers who use several local and hosted models?
- RQ2: How can the routing decision be divided into constraint filtering, suitability prediction, selection and reliability control so that each part can be tested on its own?
- RQ3: How accurately can a lightweight classifier combined with embedding retrieval predict model-specific response quality before any model is called?
- RQ4: To what extent does constrained selection reduce cost and latency relative to always using the frontier model, while keeping quality within the agreed non-inferiority margin?
- RQ5: To what extent do circuit breakers and bounded fallback raise the share of requests completed when one provider fails, and at what added latency and cost?
- RQ6: How much does each hybrid component (classification, embedding retrieval and constrained selection) contribute compared with simpler baselines?

For RQ4 and RQ6 the study will test the following hypotheses on the held-out test set, at a significance level of 0.05:

- H_0: The hybrid router's mean cost per request is not lower than that of the always-frontier baseline when quality is held within the non-inferiority margin.
- H_1: The hybrid router's mean cost per request is lower than that of the always-frontier baseline when quality is held within the non-inferiority margin.

A result that fails to reject H_0, or that shows a simpler baseline performing as well as the hybrid, will be reported as a valid finding.

1.5 Methodology Overview

The project will follow Design Science Research (Hevner et al., 2004; Peffers et al., 2007), because its main output is an artefact, the router, built to solve a stated problem and evaluated against objectives. Development will use Scrum in six two-week sprints (Schwaber & Sutherland, 2020), since the suitability predictor and selection thresholds can only be tuned once the data path, adapters and scored data are working.

The router will be written in Python with FastAPI. Prompt features will come from a sentence-embedding model and a scikit-learn classifier (Pedregosa et al., 2011; Reimers & Gurevych, 2019). The predictor will be trained and first evaluated offline on stored, scored model outputs from public routing benchmarks (Hu et al., 2024; Li et al., 2026), split by prompt and task so that no test prompt is seen in training. A small, capped live pilot will measure real latency and cost on the local and hosted models, and a mock provider will inject timeouts, rate limits, server errors and malformed responses to test fallback. Results will be compared with baselines using paired statistical tests. Chapter Three gives the details.

1.6 Significance of the Study

Practical significance. Developers and small organisations that use several models will get an open, inspectable router whose decisions are logged with reasons, together with evidence on whether it saves money at an acceptable quality. Teams with consumer GPUs will learn how far a local model can take load off paid APIs. Operators will get a tested fallback design whose retry limits and breaker settings are chosen from fault-injection experiments.

Academic significance. The study will add a controlled comparison of a hybrid predictor against simpler routing baselines on shared data, with ablations, at a time when benchmarks report that many routers do not beat the best single model (Li et al., 2026). It will also separate operational fallback from quality-based cascading in evaluation, which the literature often reports together. A negative or inconclusive result is still useful evidence for students and researchers choosing a routing method.

1.7 Scope of the Study

The project will build a routing service exposed as one OpenAI-compatible chat endpoint. It will cover:

- three or four candidate models: a small model run locally through Ollama, a mid-tier hosted model and a frontier hosted model, with an optional fourth if aligned benchmark data and the budget allow;
- hard-constraint filtering (capability, context length, cost cap, deadline and provider health);
- a suitability predictor using task classification and embedding retrieval, and a constrained selection policy;
- a reliability controller with health tracking, circuit breakers, bounded retries and fallback;
- request, decision and attempt logging in PostgreSQL, and a basic administrator dashboard;
- an evaluation harness for offline replay, live pilot timing and fault injection.

Candidate model identities will be fixed after checking that the chosen benchmark contains aligned, scored outputs for all of them. LLMRouterBench lists Qwen3-8B, Gemini-2.5-flash and GPT-5-medium as candidates in its pools, but shared coverage has not yet been confirmed (Li et al., 2026).

The project will not train or fine-tune language models, build a reinforcement-learning or ensembling router, manage conversation memory, or schedule work inside a serving engine. A quality cascade that re-asks a stronger model after a weak answer is an optional experiment and will be reported separately if time allows. The system targets developers, not end users, and will be developed on the team's two RTX 4070 laptops (a Lenovo Legion 7i and an ASUS ROG Zephyrus G16) during the 2026/2027 academic session.

1.8 Limitations of the Study

- **API budget.** Hosted calls cost money, so most quality labels will come from stored benchmark outputs rather than new calls. The live pilot will be capped, and its sample will be smaller than the offline test set. Offline results will be reported separately from live results.
- **Label quality.** Benchmark labels come from automatic scoring or LLM judges, which carry known biases (Zheng et al., 2023). The study will report which scoring method each label uses and test sensitivity where labels disagree.

- **Model and price drift.** Provider models and prices change. Each experiment will record model revisions and the price table used, so results can be read for that date.
- **Local hardware.** Two consumer laptops cannot represent a production cluster, and a quantised local model may not match the full-precision model scored in the benchmark. The local model's quality will be checked on a sample before benchmark labels are used for it.
- **Benchmark coverage.** If no benchmark offers aligned scored outputs for the chosen models, the team will choose a smaller aligned pool rather than join unrelated scores, and will report the change.

1.9 Contributions of the Project

The project is expected to produce:

- a working, open-source routing gateway with an OpenAI-compatible endpoint, adapters for local and hosted models and logged decisions;
- a hybrid suitability predictor and a constrained selection policy, with documented features, thresholds and versioning;
- a reliability controller design with retry and circuit-breaker settings chosen from fault-injection results;
- an evaluation harness and experiment protocol covering offline replay, live timing and fault injection, which other students can rerun;
- a comparative evaluation of the hybrid router against simple baselines, with an ablation of each component.

1.10 Organisation of the Report

Chapter Two reviews the literature on LLM routing, suitability prediction, cost and latency, failure handling and router evaluation, compares existing gateways and states the research gap. Chapter Three describes the methodology, requirements, system design, data, tools, evaluation plan, ethics, risks, work plan and budget. Chapter Four will report the implementation, testing and results. Chapter Five will summarise the project, state the conclusions and give recommendations and future work.

1.11 Definition of Operational Terms

- **Ablation:** an experiment that removes one component of the router (for example embedding retrieval) to measure how much that component contributes.
- **Baseline:** a simpler routing policy used for comparison, such as always using the frontier model or always using the cheapest eligible model.
- **Circuit breaker:** a mechanism that stops sending requests to a provider after repeated failures and allows a trial request after a cool-down period (Nygard, 2018).
- **Constraint filter:** the deterministic step that removes models that cannot serve a request, for example because the prompt exceeds the context window or the provider's breaker is open.
- **Hard constraint:** a limit the router must never break, such as a client's cost cap or a required capability.
- **Non-inferiority margin:** the largest drop in mean quality, relative to the always-frontier baseline, that the study will accept as "no worse".
- **Operational fallback:** re-sending a failed request (timeout, rate limit, server error or invalid response) to another eligible model. It differs from a quality cascade, which escalates after a completed but weak answer.
- **p95 latency:** the time within which 95% of requests complete.
- **Routing:** choosing which model will answer a request before any model is called (Varangot-Reille et al., 2026b).
- **Suitability:** the predicted quality of a specific model's answer to a specific prompt, written $\hat{Q}(m, q)$.
- **Time to first token (TTFT):** the time from sending a request to receiving the first token of the response (Agrawal et al., 2024a).

CHAPTER TWO

LITERATURE REVIEW

This chapter reviews the work that the project builds on. Section 2.1 explains how the literature was found and selected. Section 2.2 sets out the concepts and the theoretical lens, and Section 2.3 compares the techniques available for each part of the router. Section 2.4 reviews related work in six themes, Section 2.5 compares existing gateways, and Section 2.6 summarises the reviewed works in a table. Section 2.7 states the research gap and maps it to the objectives, and Section 2.8 closes the chapter.

2.1 Review Methodology

The review followed the step-by-step approach taught in the module: the topic was divided into sub-topics, Boolean search strings were written for each, and the results were screened against stated criteria (Kitchenham & Charters, 2007). The project title was split into nine sub-topics: routing algorithms; prompt difficulty and suitability prediction; load balancing; cost reduction; latency; error handling and fallback; testing and benchmarking; gateway and serving system design; and retrieval-based and multi-turn routing.

Table 2.1: Literature search strategy

Item	Description
Databases	Google Scholar, Semantic Scholar, arXiv, ACL Anthology, ACM Digital Library, IEEE Xplore, OpenReview and conference proceedings (NeurIPS, ICML, ICLR, MLSys, USENIX OSDI, ATC and FAST)
Search strings (examples)	("LLM" OR "large language model") AND ("routing" OR "router" OR "model selection"); ("prompt difficulty" OR "query difficulty" OR "performance prediction") AND ("LLM"); ("cascade" OR "cost-efficient") AND ("LLM inference"); ("LLM serving") AND ("SLO" OR "latency" OR "load balancing"); ("failover" OR "circuit breaker" OR "retry") AND ("LLM" OR "microservice"); ("router benchmark" OR "routing benchmark") AND ("LLM")
Period covered	2020 to 2026, plus a small number of earlier works on method and theory
Inclusion criteria	Reports a system, method or measured results on a sub-topic; published in English; traceable bibliographic record with a DOI or stable proceedings link; peer-reviewed version preferred where one exists

Exclusion criteria	Duplicates; published before 2020 (except method and theory sources); no method or results; blogs and Wikipedia; vendor pages (used only as grey literature in Section 2.5)
Studies included	90 studies retained in the review register (9 to 11 per sub-topic); 33 reviewed in depth in Section 2.4. Counts of records identified and screened before full-text reading were not recorded during the search and are reported as a limitation.

Preprints were kept where they are the only source on a sub-topic, mainly for 2026 work, and are labelled as preprints in the reference list. Where a preprint later appeared in a peer-reviewed venue, the venue version is cited. Vendor documentation for existing gateways was read on 31 October 2026 and is treated as grey literature.

2.2 Conceptual and Theoretical Framework

Routing and cascading. A router chooses a model before generation; a cascade generates with one model, judges the answer and may call another (Dekoninck et al., 2025; Moslem & Kelleher, 2026). Routing pays for one call per request but depends entirely on how well quality can be predicted from the prompt. Cascading can inspect the answer, but a hard request pays for every failed attempt before the right model answers (Chen et al., 2024a). This project is a router; the cascade is an optional comparison.

Suitability. The useful quantity is not how "hard" a prompt is in general but how well a given model will answer it. Difficulty is model-dependent: a prompt that defeats one model may be easy for another (Šakota et al., 2024; Zhu et al., 2025). The project therefore predicts $\hat{Q}(m, q)$, the expected quality of model m on prompt q, and uses difficulty and task category only as features. Predicted quality is not a probability of success unless it is calibrated, and confidence scores from LLMs are often overconfident (Xiong et al., 2024), so calibration will be measured, not assumed.

Constraints and objectives. Each request has hard constraints (required capability, context length, cost cap, deadline, provider health) and soft objectives (higher quality, lower cost, lower latency). Following Dekoninck et al. (2025) and the constrained formulation used by Šakota et al. (2024), the router first removes infeasible models and then selects among the rest. A weighted utility that trades quality against cost and latency is a comparator, not the main policy, because weights on different scales are hard to justify and can override limits a client set (Varangot-Reille et al., 2026b).

Quality, cost and latency. Quality is measured by the benchmark's own scoring (exact match, test pass or judge score). Cost is the provider's price per input and output token times the tokens used; a local model is given a declared cost basis rather than zero, following the reasoning in Griggs et al. (2024). Latency is split into gateway overhead, TTFT and total completion time, and reported at the 95th percentile, because averages hide the tail that users notice (Agrawal et al., 2024a; Zhong et al., 2024).

Reliability. Remote calls fail. Standard patterns are bounded retries with exponential backoff and jitter, timeouts, circuit breakers and fallback to an alternative (Nygard, 2018). Unbounded retries can sustain an outage (Huang et al., 2022), so every retry in this project is bounded by attempt count, remaining budget and remaining deadline.

Theoretical lens. The study uses Design Science Research, in which an artefact is built to address a relevant problem and is rigorously evaluated, with the design and the evaluation both contributing knowledge (Hevner et al., 2004). Its process model, from problem identification to communication, structures the methodology in Chapter Three (Peffers et al., 2007). Figure 2.1 shows how the concepts relate.

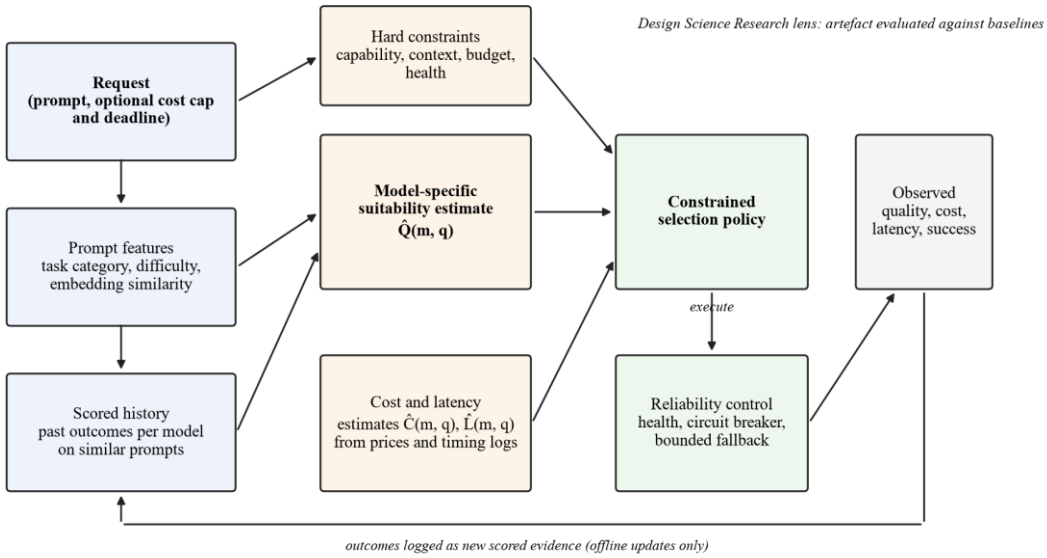


Figure 2.1: Conceptual framework of the study

2.3 Enabling Technologies and Techniques

This section compares the main options for each part of the router and states the choice made. The pattern in each table is the same: option, strength, weakness for this project, decision.

Suitability prediction. Table 2.2 compares predictor families. The gateway must decide in a few tens of milliseconds on a laptop without access to the hosted models' internals, which rules out methods that read hidden states (Zhu et al., 2025) or that call an LLM to analyse each prompt (Tang et al., 2026).

Table 2.2: Comparison of suitability prediction techniques

Technique	Strength	Weakness for this project	Decision
Hand-written rules (keywords, length)	Simple, explainable, no training data	Cannot tell which model suits a prompt; brittle across tasks	Baseline
Logistic regression or small classifier on sentence embeddings	Fast on CPU; trained from scored outputs; per-model probabilities (Šakota et al., 2024)	Needs labelled outcomes per model; calibration must be checked	Adopted (classification component)
k-nearest-neighbour retrieval of scored past prompts	Training-free; matches learned routers on several benchmarks (Varangot-Reille et al., 2026a; Shnitzer et al., 2023)	Weak when similar prompts are absent; depends on embedding quality	Adopted (retrieval component); also a baseline
Fine-tuned encoder routers (DeBERTa, RouterDC, CSCR)	Strong reported results (Chen et al., 2024b; Ding et al., 2024; Shirkavand et al., 2025)	Training cost; harder to retrain when the pool changes	Not used initially
LLM-based routers (Routoo, VDAR)	Rich features (Mohammadshahi et al., 2024; Tang et al., 2026)	Adds an LLM call and latency to every request	Not used
Hidden-state probes	Accurate on open models (Zhu et al., 2025)	Impossible for hosted APIs	Out of scope

Selection policy. Threshold routing between two models (Ding et al., 2024; Ong et al., 2025) does not extend cleanly to three or four models with cost caps and deadlines. A weighted utility $U = \alpha\hat{Q} - \beta\hat{C} - \gamma\hat{L}$ is common (Da & Kalyvianaki, 2026), but its result depends on how the terms are scaled. A constrained policy that keeps models meeting a minimum predicted quality, cost cap and deadline, then picks the cheapest, follows the linear-programming view of Dekoninck et al. (2025) and the budget-constrained assignment of Šakota et al. (2024). **Decision:** the constrained policy is the main policy, the weighted utility is a comparator, and a quality cascade is an optional experiment.

Model serving. For the local model, Ollama runs quantised models on consumer GPUs with a simple HTTP API and is supported on Windows; vLLM offers higher throughput through PagedAttention (Kwon et al., 2023) but targets Linux GPU servers and its strengths appear under concurrency the laptops will not reach. **Decision:** Ollama for the local model, with vLLM as an option if a Linux setup becomes available. Hosted models are called through their providers' APIs behind a common adapter.

Resilience patterns. Retries with exponential backoff and jitter, timeouts and circuit breakers are established patterns (Nygard, 2018). Aderaldo and Mendonça (2026) found in 222 fault-injection scenarios that the maximum number of attempts mattered most for success rate and that the best backoff varied with the failure rate, so fixed defaults are not safe. **Decision:** per-provider breakers and bounded retries, with settings chosen by fault-injection experiments in Sprint 5.

Evaluation data. RouterBench provides about 405,000 stored responses from 11 models with costs (Hu et al., 2024); LLMRouterBench provides 391,645 instances across 21 datasets and 33 models, with shared splits and simple baselines (Li et al., 2026); RouterArena adds difficulty levels and router overhead measurements (Lu et al., 2026). **Decision:** LLMRouterBench as the primary source, if aligned outputs exist for the chosen models, with RouterBench as the fallback. Licence terms for each will be checked before use.

2.4 Review of Related Works

The related works are grouped into six themes, ordered from the core prediction problem to evaluation.

2.4.1 Learned pre-generation routing between models

The first group learns to choose a model from the prompt alone. Ding et al. (2024) trained a DeBERTa router on the quality gap between a small and a large model and used a threshold to trade cost against quality, reporting up to 40% fewer calls to the large model without loss of quality when the two models were close in ability. Ong et al. (2025) trained routers on Chatbot Arena preference data and reported a cost-saving ratio of 3.66 times relative to GPT-4 while keeping 95% of its MT-Bench quality. Both route between exactly two models, and neither handles latency limits or provider failure.

Several studies extend routing to more than two models. Chen et al. (2024b) proposed RouterDC, which learns query and model embeddings with two contrastive losses so that several good models can be preferred for one query; it reached 58.54% average accuracy against 54.56% for the best single model across seven 7B to 8B models. Shirkavand et al. (2025) added cost bands to contrastive training and model "fingerprints" so that new models can be added without retraining, and reported the best area under the deferral curve on three benchmarks. These results come from accuracy-only or price-only settings with fixed model prices.

The closest structural match to this project is Šakota et al. (2024). Their method trains a small DistilBERT meta-model to predict each model's success on a prompt and then assigns prompts to models under a budget by integer programming; they report about a 63% cost reduction at the accuracy of the largest model, with a calibrated meta-model. Mohammadshahi et al. (2024) used the same split into a performance predictor and a cost-aware selector with a 7B LLM as the predictor, reaching 75.87% on MMLU against 70.6% for Mixtral at equal cost. Ding et al. (2025) added best-of-n sampling to the choice and reported a 60% cost reduction with a 0.8% quality drop against GPT-4o.

Across this theme, learned predictors can cut cost substantially on their benchmarks. However, the strongest results come from heavy predictors (Mohammadshahi et al., 2024) or fixed two-model setups (Ding et al., 2024; Ong et al., 2025), costs are static token prices, and none tests latency limits or provider failure. This project borrows the predictor-then-selector split of Šakota et al. (2024) but replaces batch assignment with per-request selection under a cost cap and deadline.

2.4.2 Embedding retrieval and historical-performance features

A second group predicts suitability from the recorded outcomes of similar past prompts. Shnitzer et al. (2023) trained per-model k-nearest-neighbour correctness predictors on HELM benchmark data and found a small average gain (accuracy 0.694 against 0.688 for the best single model) while often using smaller models; their predictors fell to 0.59 accuracy out of distribution. Zhuang et al. (2025) learned compact model embeddings from a 112-model by 36,054-question correctness matrix and showed that they beat k-NN and the best single model while routing 3,000 queries in 3.80 seconds on one GPU.

Varangot-Reille et al. (2026a) compared a learned attention-based router with plain k-NN over embeddings of past queries across six benchmarks and found that k-NN performed as well as learned routers, worked with 1% of the history, and gained nothing beyond about 40 neighbours. Wu and Silwal (2025) combined nearest-neighbour estimates with an online budget-aware rule and reported large gains in performance and throughput over eight baselines. Zhang et al. (2025) clustered prompt embeddings and scored models per cluster without training, gaining 7% accuracy at similar cost or cutting cost by 27% at equal accuracy over the best single frontier model.

These studies agree that embeddings of past prompts carry much of the routing signal, which supports including retrieval in the hybrid. They also share weaknesses: labels come from benchmark scoring, similar prompts are easy to find inside a benchmark but may be absent in real traffic, and none reports calibration or confidence intervals (Varangot-Reille et al., 2026a). Because plain k-NN is so competitive, this project treats it as a required baseline: the hybrid must beat k-NN alone to justify the classifier.

2.4.3 Cascades and joint quality–cost–latency optimisation

The third group optimises cost and quality jointly, either by calling models in sequence or by solving a selection problem. Chen et al. (2024a) built FrugalGPT, a learned cascade over 12 APIs that reported up to 98% cost saving at GPT-4 accuracy on three datasets. Aggarwal et al. (2024) used few-shot self-verification to decide when to escalate and reported over 50% cost reduction at comparable performance with as few as 50 training examples. Both pay the latency of every model tried, and both depend on in-distribution labels.

Dekoninck et al. (2025) unified routing and cascading in a linear-programming framework and showed that cascade routing beats both, by 1 to 4 points of area under the quality–cost curve, with much larger gains on some tasks. Their most important finding for this project is that the quality of the estimators decides the result: a good selection rule cannot compensate for poor quality predictions. Xiong et al. (2024) support this concern from the other direction, showing that LLM self-reported confidence is overconfident and that sampling-based consistency is needed to make it useful.

RouteBalance (Da & Kalyvianaki, 2026) is the closest prior work to joint quality–cost–latency selection. It scores model instances on predicted quality, latency and cost with a k-NN quality estimator and live queue estimates from vLLM workers, and reports 2.6 to 4.1 times lower

latency than an enhanced BEST-Route baseline at high load. It is a preprint, it uses one model family on a dedicated cluster, its quality labels are offline, and it does not cover hosted APIs, per-request deadlines or failover. Across this theme, joint optimisation is well formulated, but evaluations assume static prices and do not separate the contributions of the predictor's parts, which is what this project's ablation will measure.

2.4.4 Latency targets and deadline-aware selection

The fourth group concerns meeting latency targets, mostly in serving systems below the gateway. INFaaS (Romero et al., 2021) lets callers state accuracy, latency and cost requirements and selects a model variant per query, achieving 1.6 times fewer service-level objective (SLO) violations and up to 21.6 times lower cost, with only 2.2 to 3.5 ms of median selection overhead. Clockwork (Gujarati et al., 2020) predicts inference durations and rejects work that cannot meet its deadline, meeting a 100 ms target for 99.9999% of requests. Both serve image models with predictable run times, unlike LLM calls whose length varies with the output.

For LLMs, SLOs-Serve (Chen et al., 2025) admits requests only when predicted latency can meet their targets and routes across replicas, reporting 2.2 times more capacity per GPU. DistServe (Zhong et al., 2024) separates prompt processing from decoding to meet TTFT and per-token targets, serving up to 7.4 times more requests than vLLM-class systems. Etalon (Agrawal et al., 2024a) argues that average per-token latency hides stalls users notice and shows how to probe hosted APIs as black boxes.

The common thread is that latency should be handled as an admission decision against a stated target, with prediction error measured. INFaaS and Clockwork show the principle at the gateway level, but for predictable workloads; the LLM systems need control of the serving engine, which a gateway calling hosted APIs does not have. This project applies the admission idea at the gateway, using latency estimates from its own timing logs, and will report how often predicted latency is wrong.

2.4.5 Failure handling and fallback

The fifth group studies failures and recovery. Huang et al. (2022) analysed 21 public incident reports and found that retries sustained more than half of the metastable failures they studied, which argues for bounded retries with backoff. Yan et al. (2025) coded production incidents in a large provider's generative AI services and found that 49.8% were degradations

rather than outages and 38.3% were first detected by humans, so a router must handle slow and partial failures as well as clean errors.

Two recent preprints study LLM gateways directly. ContinuityBench (Pandey & Singh, 2026a) injected faults into a gateway that fails over between two providers over 150 synthetic conversations and found that forwarding history preserved context in 99.2% of failovers while a stateless proxy preserved none; it also uncovered a retry storm and a race condition. FailureAtlas (Pandey & Singh, 2026b) catalogued gateway failures from the issue trackers of LiteLLM, Portkey and OpenRouter and found that the worst were silent, returning a success status with an invalid body. Aderaldo and Mendonça (2026) provide a peer-reviewed method for choosing retry settings by fault injection.

The evidence on LLM gateway failover is thin: two preprints from the same authors with small samples, plus general microservice studies. None evaluates fallback that chooses the next model by predicted quality and remaining budget. This project will therefore test fallback with injected timeouts, rate limits, server errors and malformed responses, and will choose its retry and breaker settings from those tests.

2.4.6 Evaluating routers

The last group concerns how routers are evaluated. RouterBench (Hu et al., 2024) stores responses from 11 models across about 405,000 samples and plots routers on a cost–quality plane; it found that learned routers barely beat a simple non-learned router. LLMRouterBench (Li et al., 2026) re-evaluated ten routers on 21 datasets and 33 models under shared conditions and found that routers converge, many do not beat the best single model, and a large gap to the oracle remains. RouterEval (Huang et al., 2025) used over 200 million records and showed that existing routers stay far below the oracle as the pool grows.

RouterArena (Lu et al., 2026) compared 12 routers, including commercial ones, on about 8,400 queries and found that no router won on all metrics, that commercial routers were not better, and that some routers added 250 to 550 ms of overhead. Zheng et al. (2023) showed that GPT-4 as a judge agreed with human experts 85% of the time, close to the 81% agreement between humans, but with position, verbosity and self-preference biases. This matters because many routing labels are judge scores.

These evaluations set the benchmark for this project. They show that a new router must be compared against the best single model, cheap baselines and the oracle under shared splits; that router overhead must be measured; and that judge-scored labels need caution. None measures live latency on real providers or injected failures, which this project adds alongside the standard offline comparison.

2.5 Review of Existing Systems

Five existing systems were compared using their public documentation, read on 31 October 2026. OpenRouter is a hosted service that gives one API to many providers. Its provider routing chooses among providers of a named model, preferring those without recent outages and then lower prices, with latency filters based on statistics over recent minutes; its Auto Router picks a model per prompt from a task classifier and community spending data (OpenRouter, n.d.-a, n.d.-c). LiteLLM is an open-source router and proxy that balances traffic across deployments of a named model with least-busy, latency-based and cost-based strategies, plus fallbacks; its Auto Router sorts requests into complexity tiers using heuristics, an LLM classifier or adaptive selection (Dholakia, 2026; LiteLLM, n.d.-a, n.d.-b). It also supports self-hosted vLLM deployments (LiteLLM, n.d.-d).

Portkey is a gateway with weighted load balancing, fallbacks, conditional routing on request metadata, automatic retries and a circuit breaker (Portkey, n.d.-a, n.d.-c, n.d.-d). The vLLM Semantic Router is an open-source, Apache-2.0 router that extracts signals from each prompt and selects among self-hosted models, with weighted failover (Liu et al., 2026; vLLM Semantic Router Team, n.d.). Not Diamond is a closed hosted service that predicts the best model per query and lets customers train custom routers, priced per million tokens routed (Not Diamond, n.d.-a, n.d.-c, n.d.-e). Table 2.3 compares them with the proposed system.

Table 2.3: Feature comparison of existing systems and the proposed system

Feature	OpenRouter	LiteLLM	Portkey	vLLM Semantic Router	Not Diamond	Proposed system
Chooses a model per prompt	Partial (Auto Router)	Partial (Auto Router, beta)	✗ (rules on metadata)	✓	✓	✓
Learned, model-specific quality prediction	✗	✗	✗	Partial	✓ (method not published)	✓

Hard cost cap per request	Not stated	Not stated	Not stated	Not stated	✗	✓
Per-request deadline	Partial (soft latency preference)	Not stated	Not stated	Not stated	✗	✓
Fallback on provider failure	✓	✓	✓ (with circuit breaker)	✓	Not stated	✓
Local and hosted models together	✗ (hosted providers and own keys)	✓	Not stated	Partial (self-hosted)	Partial	✓
Open, inspectable routing logic	✗	✓	Partial	✓	✗	✓
Published evaluation of routing decisions against baselines	✗	✗	✗	Partial (preprint)	✗	✓ (planned)

"Not stated" means the pages read do not mention the feature, not that it is absent. The comparison shows that commercial and open-source gateways already handle fallback and, increasingly, per-prompt choice. What none of them provides is a published, measured comparison showing that its per-prompt choice beats simple baselines under hard cost and time limits. That is the system-level gap this project addresses.

2.6 Summary of Reviewed Works

Table 2.4 summarises the works reviewed in Section 2.4, grouped by theme.

Table 2.4: Summary of related works

S/N	Author(s) and year	Aim / problem	Method / technique	Dataset / tools	Key findings	Limitations / gap
1	Ding et al. (2024)	Cut large-model calls while keeping quality	DeBERTa router on quality-gap labels; threshold	MixInstruct; Llama-2, FLAN-T5, GPT-3.5	Up to 40% fewer large-model calls with no quality drop for close pairs	Two models; no latency limit or failover
2	Ong et al. (2025)	Route between a strong and a weak model	Win-probability router from preference data	Chatbot Arena; MT-Bench, MMLU, GSM8K	3.66× cost saving at 95% of GPT-4 MT-Bench quality	Two models; static prices
3	Chen et al. (2024b)	Route among several good LLMs	Query and model embeddings with dual contrastive losses	Seven 7B–8B models; MMLU, GSM8K and others	58.54% vs 54.56% for the best single model	Accuracy only; fixed pool; no cost

4	Shirkavand et al. (2025)	Cost-aware routing over changing pools	Cost-banded contrastive router; model fingerprints	EmbedLLM, MixInstruct, RouterBench	Best area under deferral curve on all three	Static cost; no failure handling
5	Šakota et al. (2024)	Cheapest model that solves each query	DistilBERT per-model success predictor; integer programming	14 HELM datasets; 4 models	About 63% cost cut at largest-model accuracy	Old single-vendor pool; no latency
6	Mohammadshahi et al. (2024)	Cheapest adequate model per prompt	LLM-based predictor and greedy selector	MMLU; about 75k prompts	75.87% vs 70.6% for Mixtral at equal cost	One benchmark; heavy predictor; preprint
7	Ding et al. (2025)	Route without over-using the largest model	Multi-head router over model and sample count	10k instructions; 8 LLMs	60% cost cut, 0.8% quality drop	One judge; static prices
8	Shnitzer et al. (2023)	Choose an LLM for a new task from benchmarks	Per-model k-NN correctness predictors	HELM (29 datasets, 18 models)	0.694 vs 0.688 accuracy; 0.59 out of distribution	Task level; small gain; preprint
9	Zhuang et al. (2025)	Compact LLM embeddings for routing	Matrix factorisation on correctness matrix	112 models × 36,054 questions	Beats k-NN and best single model	Fixed pool; correctness only
10	Varangot-Reille et al. (2026a)	Routing that generalises to new models	Attention over k nearest past queries vs plain k-NN	Six benchmarks incl. RouterBench	k-NN matches learned routers with 1% of history	Benchmark labels; no calibration
11	Wu & Silwal (2025)	Online routing under a token budget	Nearest-neighbour estimates; dual-weight rule	RouterBench, SPROUT	Large gains over eight baselines	Quality and cost only; simulated
12	Zhang et al. (2025)	Training-free quality–cost routing	Embedding clusters with per-cluster scores	6 benchmarks; 8 frontier models	+7% accuracy at similar cost; 27% cheaper at equal accuracy	Single split; no latency
13	Chen et al. (2024a)	Cut API cost while keeping accuracy	Learned cascade with answer scorer	3 datasets; 12 APIs	Up to 98% saving at GPT-4 accuracy	Sequential latency; in-distribution labels
14	Aggarwal et al. (2024)	Mix small and large models through APIs	Self-verification with threshold or POMDP router	5 context-based QA datasets	Over 50% cost reduction at comparable quality	Context tasks only; sequential latency
15	Dekoninck et al. (2025)	Unify routing and cascading	Linear-programming cascade routing	RouterBench, SWE-Bench and others	1–4 AUC points over both; estimator quality decisive	Static costs; no live latency
16	Xiong et al. (2024)	Benchmark black-box LLM confidence	Prompting, sampling and	8 datasets; 5 model families	Verbalised confidence is overconfident	Needs generation per query

			aggregation; ECE, AUROC			
17	Da & Kalyvianaki (2026)	Choose model and instance under load	Weighted quality–latency–cost score; k-NN quality	13 instances, Qwen2.5 3B–72B	2.6–4.1× faster than enhanced BEST-Route at high load	One family; offline labels; preprint
18	Romero et al. (2021)	Remove manual model-variant choice	Model-less API; per-query variant selection	175 variants on AWS	1.6× fewer SLO violations; up to 21.6× lower cost	Image models; no prompt-level quality
19	Gujarati et al. (2020)	Predictable tail latency	Central scheduler with predicted durations	24 V100 GPUs; Azure traces	100 ms target met for 99.9999% of requests	Deterministic models only
20	Chen et al. (2025)	Meet per-stage SLOs for LLM apps	Token-budget scheduler; SLO-attainability routing	vLLM, 4×A100	2.2× per-GPU capacity	Needs profiling; preprint
21	Zhong et al. (2024)	Goodput under TTFT and per-token targets	Prefill/decode disaggregation	OPT 13B–175B on A100s	Up to 7.4× more requests	Engine level; no faults
22	Agrawal et al. (2024a)	User-facing latency evaluation	Fluidity index; black-box API probing	vLLM, Sarathi-Serve, hosted APIs	Tail token latency 3× worse while mean per-token time matched	Preprint; targets chosen empirically
23	Huang et al. (2022)	What sustains metastable failures	Incident survey; load-capacity model	21 incident reports	Retries sustain over half of incidents	Not LLM specific
24	Yan et al. (2025)	Incidents in GenAI cloud services	Manual coding of incidents	One provider, 2020–2024	49.8% degradations; 38.3% human-detected	One provider; preprint
25	Pandey & Singh (2026a)	Continuity under provider failover	Fault injection; history-forwarding proxy	150 synthetic conversations	99.2% context preserved vs 0% stateless	Preprint; synthetic; two providers
26	Pandey & Singh (2026b)	Classify gateway failure modes	Taxonomy from issue trackers; reproductions	LiteLLM, Portkey, OpenRouter issues	Worst failures are silent (success status)	Five entries; preprint
27	Aderaldo & Mendonça (2026)	Evaluate retry and circuit-breaker settings	Declarative fault-injection scenarios	222 scenarios on Kubernetes	Attempt limit drives success; best backoff varies	Microservices; not LLMs
28	Hu et al. (2024)	Standard router benchmark	Stored responses; cost–quality plane	405k samples; 11 LLMs	Learned routers barely beat a simple router	No latency; partly judge-scored
29	Li et al. (2026)	Unified current routing benchmark	Shared framework; ten routers	21 datasets; 33 models; 391,645 instances	Many routers do not beat best single model	Approximate latency only

30	Huang et al. (2025)	Router benchmark and pool-size effects	Classification labels over growing pools	8,500+ LLMs; 200M+ records	Routers stay far below oracle	No cost or latency
31	Lu et al. (2026)	Open platform for comparing routers	Difficulty-graded dataset; arena score	About 8,400 queries; 12 routers	No router wins all metrics; 250–550 ms overhead for some	Router overhead only
32	Zheng et al. (2023)	Can LLM judges match human preference?	MT-Bench; pairwise judging	80 questions; 3k expert votes	85% GPT-4–human agreement vs 81% human–human	Judge biases; small set
33	Moslem & Kelleher (2026)	Map routing and cascading	Survey with when/what/how framework	Reported results only	Evaluations not comparable; four failure modes	No re-evaluation

2.7 Research Gap and Justification of the Study

Reading the limitations column of Table 2.4 from top to bottom, four gaps emerge. First, learned and retrieval-based predictors are each shown to work, and plain k-NN is often as good as learned routers, but no study tests whether combining a lightweight classifier with embedding retrieval improves on either alone under the same data and splits (Shnitzer et al., 2023; Varangot-Reille et al., 2026a; Zhuang et al., 2025). Second, selection under hard per-request cost caps and deadlines across local and hosted models is rarely evaluated: most methods optimise average cost on static prices, and the one joint quality–cost–latency study is a single-family preprint (Da & Kalyvianaki, 2026; Dekoninck et al., 2025; Šakota et al., 2024). Third, fallback for LLM gateways has thin evidence, consisting of two small preprints and general microservice studies, and none chooses the fallback model by predicted quality and remaining budget (Aderaldo & Mendonça, 2026; Pandey & Singh, 2026a, 2026b). Fourth, shared-condition benchmarks report that many routers fail to beat the best single model, yet most papers do not compare against simple baselines or separate each component's contribution (Hu et al., 2024; Li et al., 2026; Lu et al., 2026).

The project addresses these gaps together. This project investigates whether combining lightweight classification and embedding-derived suitability features with constrained cost–quality–latency selection improves on simpler routing baselines. Operational fallback is evaluated separately for resilience. The contribution is the decision policy, its integration and its empirical evaluation; the project does not claim a new learning algorithm or to be the first system of its kind.

Table 2.5: Mapping of research gaps to project objectives

Gap identified	Evidence (studies)	Objective that addresses it
No test of whether a classifier combined with embedding retrieval beats either alone	Shnitzer et al. (2023); Varangot-Reille et al. (2026a); Zhuang et al. (2025)	Objectives 3 and 6
Selection under hard per-request cost caps and deadlines across local and hosted models is rarely evaluated	Šakota et al. (2024); Dekoninck et al. (2025); Da & Kalyvianaki (2026)	Objectives 2 and 4
Thin evidence on LLM gateway fallback; no quality- and budget-aware fallback choice	Pandey & Singh (2026a, 2026b); Aderaldo & Mendonça (2026); Huang et al. (2022)	Objective 5
Routers often not compared with simple baselines; component contributions not isolated	Hu et al. (2024); Li et al. (2026); Lu et al. (2026)	Objective 6
Existing gateways publish no evaluation of their per-prompt decisions	Table 2.3 (grey literature)	Objectives 1 and 6

2.8 Chapter Summary

The review found that the building blocks for prompt-aware routing are mature. Small predictors can estimate model-specific quality from the prompt, embedding retrieval of past outcomes is a strong and cheap signal, constrained formulations of the cost–quality trade-off exist, and resilience patterns for remote calls are well understood. It also found that these blocks are rarely combined and evaluated together under hard per-request limits, that evidence on LLM gateway fallback is thin, and that shared benchmarks often show complex routers failing to beat simple ones. Existing gateways route and fail over but publish no evaluation of their routing decisions. The next chapter describes how the project will build and evaluate a router that fills this gap.

CHAPTER THREE

METHODOLOGY, SYSTEM ANALYSIS AND DESIGN

This chapter describes how the project will be carried out. It covers the research and development methodology, requirements elicitation, the analysis of current practice and the proposed system, the requirements specification, the system design, the data plan, tools, evaluation plan, ethical considerations, risks, work plan, budget and expected outcomes. It is written so that another team with similar skills could repeat the work.

3.1 Research and Development Methodology

Research design. The project will use Design Science Research (DSR), which builds an artefact to solve a relevant problem and evaluates it rigorously (Hevner et al., 2004). DSR suits this project because its main output is a working router and its research questions are about whether that artefact performs better than alternatives. The six activities of Peffers et al. (2007) map onto the project as shown in Figure 3.1: the problem is identified in Chapter One and the gaps in Section 2.7; objectives with measurable targets are defined in Section 1.3 and Table 3.2; the artefact is designed and built in sprints; it is demonstrated on benchmark replay and a capped live pilot; it is evaluated against baselines and under fault injection; and the results are communicated in the final report, code repository and experiment protocol. The evaluation itself will be a controlled offline experiment with paired comparisons (Wohlin et al., 2012).

Development model. Development will use Scrum with six two-week sprints (Schwaber & Sutherland, 2020). Scrum fits because the predictor and selection thresholds can only be tuned after the gateway, adapters and data pipeline are working, and early results may change the design. Solomon will act as Scrum Master and coordinate with the supervisor, who will act as product owner for priorities. The team will hold a planning meeting at the start of each sprint, short stand-ups twice a week, and a review and retrospective at the end of each sprint. All code will live in one GitHub repository, with pull requests reviewed by a second member before merging.

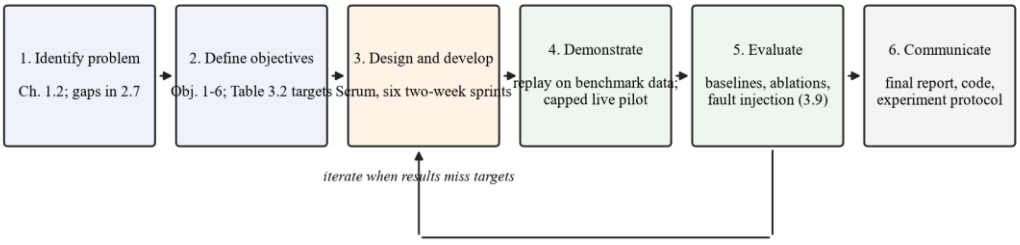


Figure 3.1: Design Science Research activities applied to the project

3.2 Requirements Elicitation

The system's users are developers and administrators who integrate LLMs into software, not members of the public. Requirements will be gathered from four sources without recruiting human participants, a decision recorded by the team and to be confirmed with the supervisor:

- **Review of existing systems.** The documentation of OpenRouter, LiteLLM, Portkey, vLLM Semantic Router and Not Diamond (Section 2.5) will be analysed for features, configuration options and documented limits.
- **Document analysis.** Provider API references will be analysed for request formats, error codes, rate-limit behaviour, token pricing and timeouts, and the OpenAI-compatible chat API will be used as the interface contract.
- **Literature.** The gaps and evaluation practices in Chapter Two will be turned into requirements, for example that routing overhead must be measured (Lu et al., 2026) and that silent failures must be detected (Pandey & Singh, 2026b).
- **Failure catalogues.** Gateway issue reports and incident studies (Pandey & Singh, 2026b; Yan et al., 2025) will be used to list failure scenarios the reliability controller must handle.

Wahida and Kabiru will extract candidate requirements into a shared sheet, recording the source of each. The whole team will then group them, write them as user stories with acceptance criteria, and prioritise them with MoSCoW in the Sprint 1 review. If the supervisor asks for user input, a short questionnaire for five to eight student and junior developers will be added, with an information sheet and consent form placed in Appendix C.

3.3 Analysis of the Existing System

3.3.1 Description of the Existing System

In current practice a developer chooses one model, writes its name into the code or configuration, and sends every request to it through the provider's SDK or an OpenAI-compatible gateway. If a gateway is used, it may balance load across copies of that model and fall back to a named alternative after an error (LiteLLM, n.d.-a; Portkey, n.d.-b). Where per-prompt routing is offered, the developer enables a vendor feature and accepts its decisions without access to an evaluation of them (Dholakia, 2026; OpenRouter, n.d.-c). Cost is reviewed after the fact on the provider's billing page.

3.3.2 Limitations of the Existing System

- Every request pays the price of the chosen model, even when a cheaper model would answer as well (Ding et al., 2024; Ong et al., 2025).
- Choosing a small model to save money loses quality on hard requests, and the developer cannot tell which requests were affected.
- There is no per-request cost cap or deadline that the gateway enforces before sending the request.
- Fallback targets are fixed in configuration, so a failed request may go to a model that is more expensive than needed or too weak for the task.
- Retries without bounds can amplify an outage (Huang et al., 2022), and silent failures with a success status pass through unnoticed (Pandey & Singh, 2026b).
- Vendor per-prompt routing is closed or unevaluated, so teams cannot verify that it saves money without losing quality.
- Local models on the team's own GPUs are usually managed separately from hosted APIs.

3.4 The Proposed System

3.4.1 Description of the Proposed System

The proposed system is a routing gateway that exposes one OpenAI-compatible chat endpoint. A client sends a request and may add a cost cap and a deadline. The gateway removes models that cannot serve the request, predicts each remaining model's quality, cost and latency for this prompt, selects the cheapest model expected to meet the quality threshold within the limits, and calls it. If the call fails in a retryable way, the reliability controller updates the provider's health,

re-runs selection on the remaining eligible models and tries again within bounded attempts. Every decision is logged with its predictions and reason, and the response carries routing metadata. An administrator registers models and prices, sets policy thresholds and reviews usage on a dashboard. An evaluator runs replay and fault-injection experiments through the same code.

The system has five modules: the gateway API, the constraint filter, the prompt analyser and suitability predictor, the selection policy, and the reliability controller with provider adapters. A PostgreSQL database stores the model registry, requests, decisions, attempts, health events and quality scores.

3.4.2 Justification for the Proposed System

Table 3.1: Limitations of current practice and the proposed response

Limitation	Response in the proposed system
Every request pays the chosen model's price	Per-prompt selection of the cheapest model predicted to meet the quality threshold
Developer cannot see which requests lost quality	Each decision logged with predicted quality and reason; quality scored on evaluation sets
No enforced cost cap or deadline	Hard constraint filter rejects or excludes models whose estimated cost or latency exceeds the client's limits
Fixed fallback targets	Fallback re-runs selection on remaining eligible models within the remaining budget and time
Unbounded retries and silent failures	Bounded attempts, backoff with jitter, circuit breakers and response validation
Vendor routing unevaluated	Open code and a published evaluation against baselines with ablations
Local and hosted models managed separately	One adapter interface for Ollama and hosted APIs behind the same endpoint

3.5 Requirements Specification

The requirements below are preliminary and will be confirmed after elicitation in Sprint 1. Their IDs will be reused in the design traceability matrix (Section 3.6.8) and in the test cases in Chapter Four.

3.5.1 Functional Requirements

Table 3.2: Functional requirements

ID	Requirement	Actor	Priority
FR-01	The system shall accept chat requests through an OpenAI-compatible endpoint and return responses in the same format.	Client developer	Must
FR-02	The system shall authenticate each client with an API key and reject requests with a missing or invalid key.	Client developer	Must
FR-03	The system shall accept an optional maximum cost and deadline per request.	Client developer	Must
FR-04	The system shall exclude models that fail a hard constraint (capability, context length, cost cap, deadline or open circuit breaker).	System	Must
FR-05	The system shall classify each prompt by task category and estimate its difficulty.	System	Must
FR-06	The system shall retrieve the k most similar scored prompts and their per-model outcomes.	System	Must
FR-07	The system shall predict quality, cost and latency for each eligible model.	System	Must
FR-08	The system shall select the cheapest eligible model whose predicted quality meets the threshold and whose estimates meet the request's limits.	System	Must
FR-09	The system shall reject a request with a reason code when no model is feasible.	System	Must
FR-10	The system shall retry a retryable failure on the next selected model within limits on attempts, budget and deadline.	System	Must
FR-11	The system shall maintain a circuit breaker per model endpoint with closed, open and half-open states.	System	Must
FR-12	The system shall validate each response and treat malformed or empty bodies as failures.	System	Must
FR-13	The system shall log each request, decision, attempt and outcome with timestamps, tokens and cost.	System	Must
FR-14	The system shall return routing metadata (chosen model, reason, attempts) with each response.	Client developer	Should

FR-15	The system shall allow an administrator to register models, prices and capabilities and set policy thresholds.	Administrator	Should
FR-16	The system shall provide a dashboard of request volume, cost, latency, fallback rate and model health.	Administrator	Should
FR-17	The system shall replay a stored dataset through any routing policy and export the results.	Evaluator	Must
FR-18	The system shall inject timeouts, rate limits, server errors and malformed responses through a mock provider.	Evaluator	Must
FR-19	The system shall support an optional quality cascade that escalates after a low-scoring answer.	System	Could
FR-20	The system shall learn routing weights online from live traffic.	System	Won't (this project)

3.5.2 Non-Functional Requirements

The targets below are proposed and will be confirmed with the supervisor and frozen, with a recorded date, before any test results are inspected. The quality non-inferiority margin will be set from the variance observed in the Sprint 3 pilot and frozen at the same point.

Table 3.3: Non-functional requirements

ID	Category	Requirement	Measure / target
NFR-01	Performance	Routing overhead added by the gateway (excluding model time)	p95 ≤ 100 ms on Machine 1 at 10 concurrent requests
NFR-02	Performance	Deadline adherence for requests with a deadline	≥ 95% of completed requests finish within their deadline
NFR-03	Cost	Mean cost per request versus the always-frontier baseline	Lower, at quality within the non-inferiority margin (H_1)
NFR-04	Reliability	Requests completed when one provider fails	≥ 99% complete through fallback
NFR-05	Reliability	Retry bound	No request makes more than 3 attempts; none exceeds its cost cap
NFR-06	Accuracy	Calibration of predicted quality	Expected calibration error reported on the test set with a 95% confidence interval

NFR-07	Security	Credentials and transport	API keys stored as salted hashes; provider keys in environment variables; TLS for hosted calls
NFR-08	Privacy	Prompt storage	Raw prompts not stored in production logs; only hashes and features
NFR-09	Maintainability	Test coverage of selection and reliability modules	≥ 80% line coverage
NFR-10	Compatibility	Client compatibility	Works unchanged with the official OpenAI Python client by changing the base URL

3.5.3 Use Case Model

Figure 3.2 shows the actors and use cases. The client developer sends requests, sets limits and reads routing metadata. The administrator registers models and prices, configures policy and fallback settings and views the dashboard. The evaluator runs replay and fault-injection experiments. Model providers are secondary actors that receive calls. "Send chat request" always includes model selection and, when a call fails, fallback.

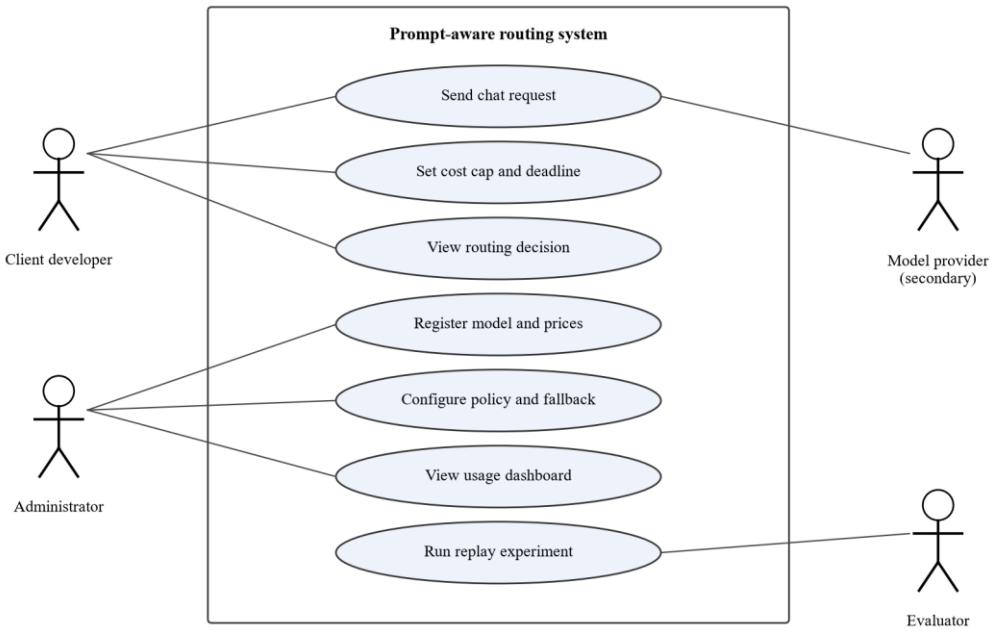


Figure 3.2: Use case diagram

Table 3.4: Use case description for "Send chat request"

Field	Description

Name	Send chat request (UC-01)
Actor	Client developer (primary); model provider (secondary)
Preconditions	Client holds a valid API key; at least one model is registered and its breaker is not open
Main flow	1. Client sends a chat request with optional cost cap and deadline. 2. Gateway authenticates and validates it. 3. Constraint filter removes ineligible models. 4. Predictor estimates quality, cost and latency for each eligible model. 5. Policy selects the cheapest feasible model. 6. Adapter calls the model. 7. Gateway validates the response, logs the outcome and returns it with routing metadata.
Alternative flows	3a. No eligible model: return an error with a reason code. 6a. Retryable failure: record health, re-run step 5 on remaining models while attempts, budget and time remain. 6b. Non-retryable failure or limits exhausted: return an error with the attempt history.
Postconditions	Request, decision and attempts are stored; cost is charged to the client's usage

3.6 System Design

3.6.1 System Architecture

The system uses a layered, modular architecture inside one service: an API layer (gateway), a decision layer (constraint filter, prompt analyser, suitability predictor and selection policy), an execution layer (reliability controller and adapters) and a data layer (PostgreSQL). A single service is preferred over microservices because the routing decision must add little latency (NFR-01) and the team is small; separating the layers by interface still lets each be unit-tested and replaced, for example to swap the predictor during ablation. Figure 3.3 shows the architecture and its deployment on the two laptops. Machine 1 runs the router, database and local model; Machine 2 runs the evaluation harness, load generator and mock provider so that test traffic does not compete with the router for resources.

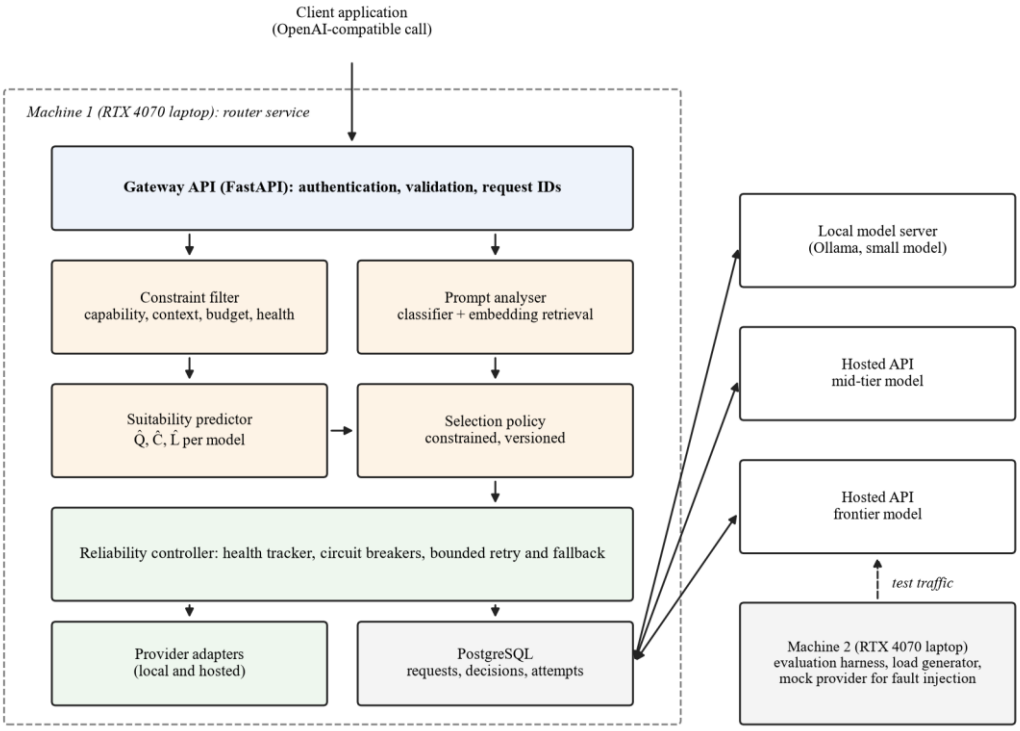


Figure 3.3: System architecture and deployment

3.6.2 Structural Design (Class Diagram)

The main classes follow the modules. *Gateway* receives requests and coordinates the flow. *ConstraintFilter* holds the hard rules and returns eligible models. *PromptAnalyser* computes the embedding, task category and difficulty. *SuitabilityPredictor* combines classifier output and retrieved neighbour outcomes into $\hat{Q}$ per model, and *CostEstimator* and *LatencyEstimator* produce $\hat{C}$ and $\hat{L}$. *SelectionPolicy* is an interface with implementations for the constrained policy, the weighted utility and each baseline, which lets the evaluator swap policies without changing other code. *ReliabilityController* owns one *CircuitBreaker* per model endpoint and a *RetryPolicy*. *ProviderAdapter* is an interface implemented by *OllamaAdapter*, *HostedApiAdapter* and *MockProviderAdapter*. *Repository* classes write to the database. The full class diagram with attributes and methods will be added in the final report.

3.6.3 Behavioural Design (Sequence and Activity Diagrams)

For a normal request, the gateway calls the constraint filter, then the prompt analyser, predictor and policy in sequence, then the reliability controller, which calls the chosen adapter and records the attempt. For a failed request, the controller records the failure on the breaker, asks the

policy to select again from the models that remain eligible with the reduced budget and deadline, and calls the next adapter. Figure 3.5 in Section 3.6.5 shows this flow as an activity diagram. Sequence diagrams for the normal and fallback paths will be included in the final report.

3.6.4 Database Design

Figure 3.4 shows the entity relationship diagram. A provider offers many models. A client sends many requests; each request has one routing decision, which leads to one or more attempts, each against one model. Each attempt may receive one quality score during evaluation. Health events record breaker state changes per model. The schema is in third normal form: each table has a single-column key, every non-key attribute depends on the whole key, and no non-key attribute depends on another (for example, prices are stored on MODEL, not repeated on ATTEMPT; the cost on ATTEMPT is the charged amount at the time of the call, kept because prices change). A full data dictionary will be given in the final report.

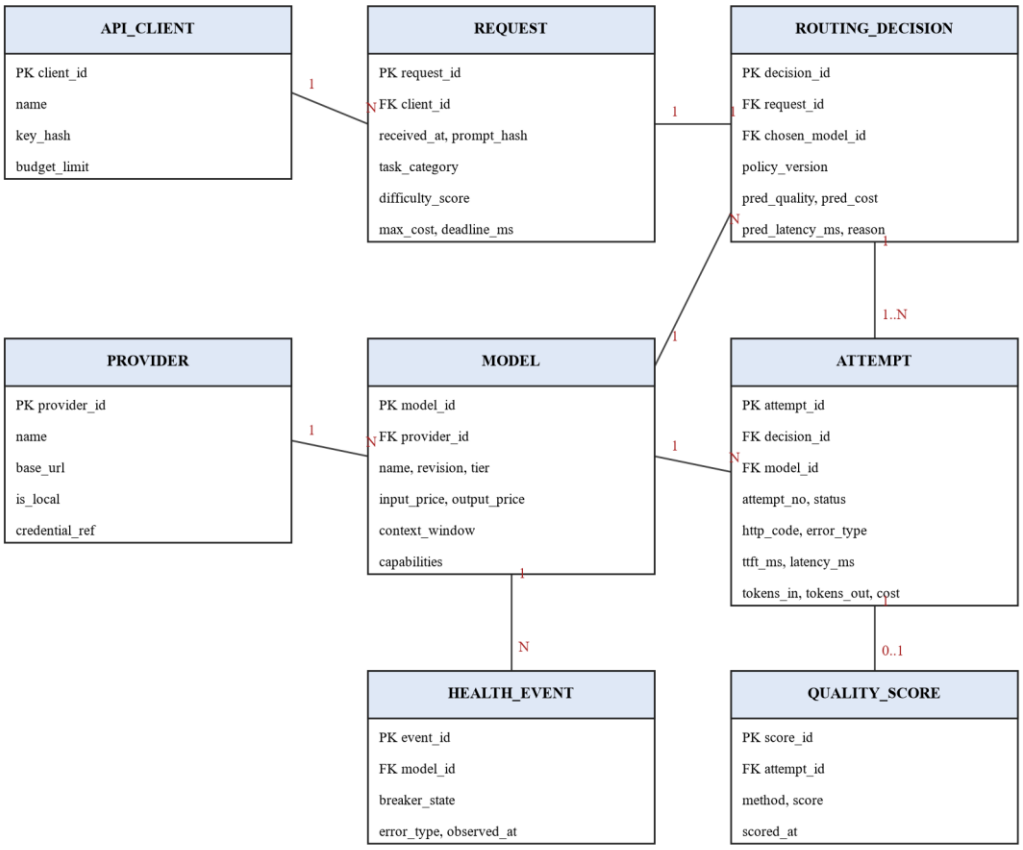


Figure 3.4: Entity relationship diagram

Table 3.5: Main relational tables (draft)

Table	Key fields	Purpose
provider	provider_id (PK), name, base_url, is_local, credential_ref	Model hosts; credential_ref names an environment variable, never the key itself
model	model_id (PK), provider_id (FK), name, revision, tier, input_price, output_price, context_window, capabilities	Registry of candidate models
api_client	client_id (PK), name, key_hash, budget_limit	Clients of the gateway
request	request_id (PK), client_id (FK), received_at, prompt_hash,	One row per incoming request

	task_category, difficulty_score, max_cost, deadline_ms	
routing_decision	decision_id (PK), request_id (FK), chosen_model_id (FK), policy_version, pred_quality, pred_cost, pred_latency_ms, reason	The decision and its predictions
attempt	attempt_id (PK), decision_id (FK), model_id (FK), attempt_no, status, http_code, error_type, ttft_ms, latency_ms, tokens_in, tokens_out, cost	Each call made, including fallbacks
health_event	event_id (PK), model_id (FK), breaker_state, error_type, observed_at	Breaker state changes
quality_score	score_id (PK), attempt_id (FK), method, score, scored_at	Evaluation scores for answers

3.6.5 Algorithm and Model Design

Features and predictor. Each prompt q is embedded with a sentence-embedding model (Reimers & Gurevych, 2019). A logistic-regression classifier on the embedding predicts the task category, and a second set of per-model logistic-regression heads predicts the probability that each model m answers acceptably (Pedregosa et al., 2011). In parallel, the k nearest prompts in the training history are retrieved by cosine similarity, and their recorded scores for model m are averaged with similarity weights to give a retrieval estimate. The hybrid estimate is

$$\hat{Q}(m, q) = w \cdot p_clf(m, q) + (1 - w) \cdot r_kNN(m, q),$$

where p_clf is the classifier probability, r_kNN the retrieval estimate and w a weight tuned on the validation split. The estimator form, k and w will be chosen on validation data only. Estimated cost is $\hat{C}(m, q) = n_in \cdot price_in(m) + \hat{n}_out(m, q) \cdot price_out(m)$, where n_in is the counted input tokens and $\hat{n}_out$ the median output length of the retrieved neighbours for model m. Estimated latency $\hat{L}(m, q)$ is the 95th percentile of recent completion times for model m at similar output lengths, taken from the attempt log.

Selection. Let E be the models that pass the hard constraints, τ the minimum predicted quality, c the request's cost cap and d its deadline. The policy selects

$$m^* = \operatorname{argmin} \hat{C}(m, q) \text{ over } m \in E \text{ such that } \hat{Q}(m, q) \geq \tau,\ \hat{C}(m, q) \leq c \text{ and } \hat{L}(m, q) \leq d,$$

breaking ties by higher $\hat{Q}$. If no model meets τ, the policy chooses the eligible model with the highest $\hat{Q}$ that still meets c and d, and marks the decision as below threshold. If no model meets c and d, the request is rejected with a reason code. The threshold τ will be swept on the validation split to trace the quality–cost curve, and the operating value fixed before testing.

Fallback. On a timeout, rate limit, server error or invalid response, the controller records a failure on that model's breaker. A breaker opens after a set number of consecutive failures, rejects calls during a cool-down, and lets one trial request through when half-open. The controller then removes the failed model, subtracts the cost already consumed from the cap and the elapsed time from the deadline, and re-runs selection. At most three attempts are made per request (NFR-05), with exponential backoff and jitter between attempts to the same provider. Failure thresholds, cool-down times and backoff values will be chosen from the fault-injection experiments in Sprint 5. Figure 3.5 shows the complete algorithm.

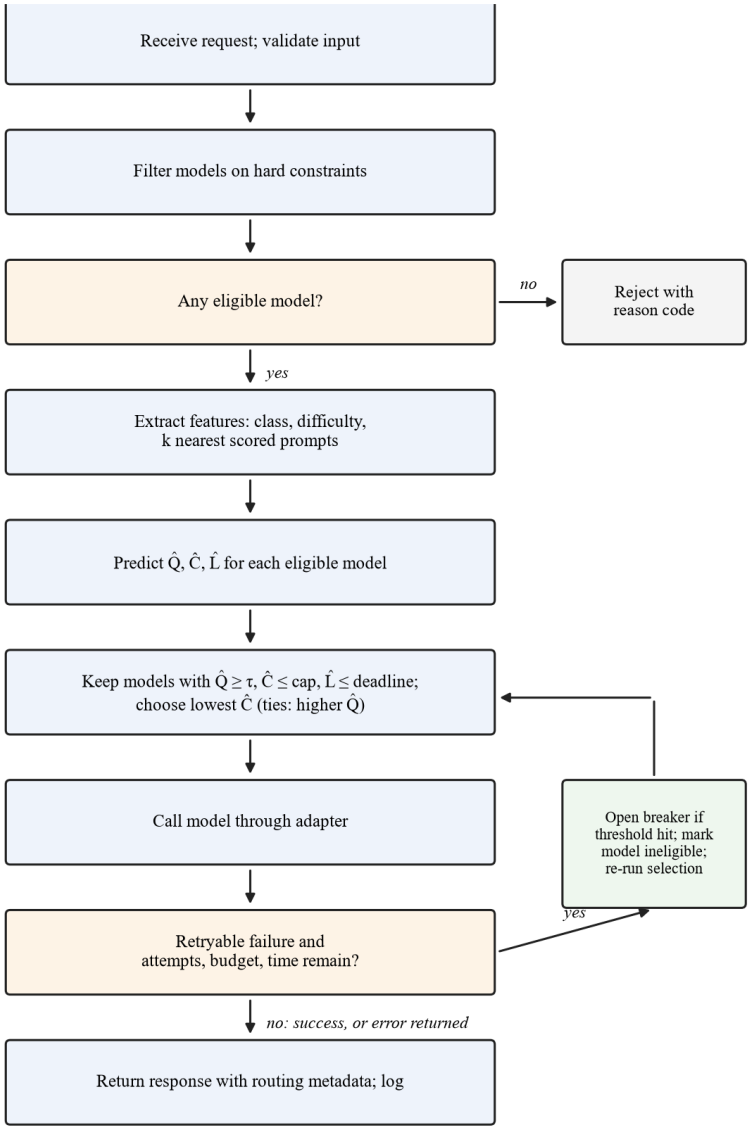


Figure 3.5: Routing and fallback algorithm

3.6.6 User Interface Design

The system's main interface is the API. The administrator dashboard is a secondary, read-mostly interface with four views: an overview with request count, spend, p95 latency and fallback rate; a request log showing the chosen model and the reason for each decision; a model page with prices, capabilities and breaker state; and an experiments page listing replay runs and their quality–cost curves. Figure 3.6 shows the overview wireframe with sample data. The design will use plain-language reasons ("cheapest feasible", "fallback after timeout"), consistent layouts across pages, text labels alongside colour for breaker states, and keyboard-accessible controls.

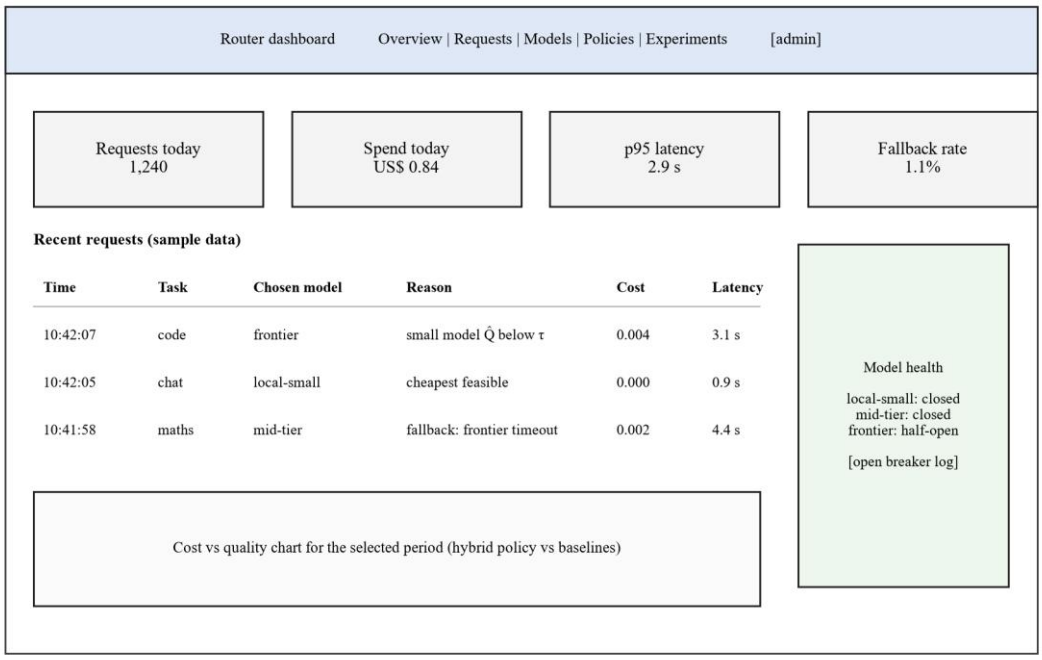


Figure 3.6: Wireframe of the administrator dashboard overview (sample data)

3.6.7 Security and Privacy Design

Clients authenticate with API keys stored as salted hashes. Provider credentials are read from environment variables and never written to the database or logs. Hosted calls use TLS. Inputs are validated against the OpenAI request schema and size limits before processing, and dashboard pages require an administrator login. Production logs will store a hash and derived features of each prompt rather than the raw text, and evaluation data will come from public benchmarks, so no personal data is expected to be processed. If any personal data is processed later, the design will follow the Nigeria Data Protection Act 2023, including purpose limitation and data minimisation (Federal Republic of Nigeria, 2023).

3.6.8 Requirements Traceability

Table 3.6: Draft traceability from requirements to design and tests

Requirement	Design element	Planned test
FR-01, NFR-10	Gateway API (Figure 3.3)	Call the endpoint with the OpenAI Python client
FR-04, FR-09	Constraint filter; reject branch (Figure 3.5)	Requests exceeding context, cost cap or deadline

FR-05 to FR-07	Prompt analyser and suitability predictor	Predictor accuracy and calibration on the test split
FR-08	Constrained selection policy	Replay comparison against baselines
FR-10 to FR-12, NFR-04, NFR-05	Reliability controller, circuit breaker, response validation	Fault-injection scenarios with the mock provider
FR-13, FR-14	Repository classes; attempt and decision tables (Figure 3.4)	Log completeness check after replay
NFR-01, NFR-02	Gateway and latency estimator	Load test at 10 concurrent requests from Machine 2

Test IDs will be assigned when the test plan is written in Sprint 2.

3.7 Data Collection and Preparation

Sources. The primary source will be LLMRouterBench, which provides stored responses and scores for 33 models across 21 datasets (Li et al., 2026); RouterBench, with 11 models and about 405,000 samples, is the fallback (Hu et al., 2024). The team will first check that the chosen candidate models have aligned, scored outputs on the same prompts. Unrelated scores will not be joined by benchmark name. Code and data licence terms will be checked for each source, and raw benchmark data will not be redistributed.

Labels. For each prompt and model the label is the benchmark's score, converted to a binary "acceptable" label for the classifier heads using the benchmark's own correctness rule, and kept as a continuous score for retrieval. The scoring method of each label (exact match, unit test or LLM judge) will be recorded, given the biases of judge scores (Zheng et al., 2023).

Local model check. The quantised local model may not match the full-precision model scored in the benchmark. A stratified sample of about 300 prompts will be run on the local model and scored with the benchmark's method; if agreement with the stored labels is poor, the local model will be labelled from its own outputs on that sample only and the limitation reported.

Splits. Prompts will be split 60/20/20 into training, validation and test sets, stratified by dataset and grouped so that near-duplicate prompts (cosine similarity above 0.95) fall in the same split. The retrieval index will contain training prompts only. All tuning (k, w, τ and baseline

settings) will use the validation set; the test set will be used once, after targets are frozen. A second test on datasets held out entirely from training will measure generalisation.

Live pilot data. A capped live pilot of a few hundred prompts drawn from the test split will measure real latency, token usage and cost on the local and hosted models. Prompts are public benchmark items, so no personal data is involved.

3.8 Tools and Technologies

Table 3.7: Tools and technologies

Tool / technology	Purpose	Justification
Python 3.12	Main language	One language for the gateway, predictor and analysis; strong ML libraries
FastAPI	Gateway API	Asynchronous request handling; automatic request validation; OpenAPI documentation
sentence-transformers	Prompt embeddings	Small embedding models run on CPU or GPU in milliseconds (Reimers & Gurevych, 2019)
scikit-learn	Classifier and calibration	Logistic regression and calibration tools; fast to retrain (Pedregosa et al., 2011)
FAISS or NumPy	Nearest-neighbour search	Exact or approximate cosine search over the training index
Ollama	Local model serving	Runs quantised models on the RTX 4070 laptops under Windows with an HTTP API
Hosted model APIs	Mid-tier and frontier models	Required for the mixed pool; called through a common adapter
PostgreSQL	Relational data	Normalised schema for requests, decisions and attempts (Section 3.6.4)
React	Administrator dashboard	Component-based UI; team familiarity
pytest, Locust or k6	Unit, integration and load tests	Coverage for NFR-09; concurrent load for NFR-01
Git and GitHub	Version control and reviews	Pull-request reviews; issue tracking per sprint

draw.io, Python (matplotlib)	Diagrams and charts	Figures and quality–cost plots

3.9 Evaluation Plan

Testing will be done at four levels: unit tests for the filter, predictor, policy, breaker and retry logic; integration tests for the path from gateway to adapter to database; system tests on replay and live pilot traffic; and acceptance tests derived from the user stories. Table 3.8 sets out the evaluation of each objective.

Table 3.8: Evaluation plan by objective

Objective	Method	Metric	Target / test
1. Requirements	Review of the requirements against sources and user stories	Requirements with source, ID and priority; supervisor sign-off	All Must requirements traced to a source
2. Design	Design review; traceability matrix	Requirements traced to design elements	100% of Must requirements traced
3. Predictor (RQ3)	Held-out test split; held-out datasets	AUROC and accuracy of acceptable-answer prediction; expected calibration error	Report with 95% bootstrap confidence intervals; compare with k-NN alone and classifier alone
4. Selection (RQ4)	Paired replay of the test split through each policy	Mean quality, mean cost, p95 latency, constraint violations	Quality within the non-inferiority margin; lower cost than always-frontier (Wilcoxon signed-rank, one-sided, $\alpha = 0.05$); acceptable/unacceptable outcomes compared with McNemar's test
5. Reliability (RQ5)	Fault injection with the mock provider: timeouts, 429s, 5xx, malformed bodies, slow responses	Completion rate, added latency, added cost, attempts per request	$\geq$ 99% completion with one provider failing (NFR-04); $\leq$ 3 attempts (NFR-05)
6. Ablation (RQ6)	Replay with components removed	Change in cost at fixed quality; area under quality–cost curve	Report each component's effect with confidence intervals
Overhead	Load test from Machine 2 at 10 concurrent requests	Gateway overhead p95; deadline adherence	p95 $\leq$ 100 ms (NFR-01); $\geq$ 95% within deadline (NFR-02)

Baselines. Every comparison will include: always the frontier model; always the cheapest eligible model; rule-based routing by task category; k-NN retrieval only; classifier only; the

weighted-utility policy; and the oracle (the cheapest model that answered acceptably, computed from stored labels) as an upper bound. A simpler baseline matching the hybrid is a valid result and will be reported as such.

Statistics. Because all policies are run on the same prompts, comparisons are paired. Binary outcomes (acceptable or not) will use McNemar's test, and per-request cost and latency will use the Wilcoxon signed-rank test (Wohlin et al., 2012). Non-inferiority will be judged by whether the lower bound of the 95% confidence interval for the quality difference lies above the negative margin. Live pilot results will be reported separately from offline replay because they use fewer prompts and real network conditions.

3.10 Ethical Considerations

The project does not plan to recruit human participants, so no consent process is required for the core evaluation. The team has recorded the position that formal ethical approval is not needed; this will be confirmed in writing with the supervisor before Sprint 1, and if approval is required the team will apply to the university's research ethics committee before any data collection. If a requirements questionnaire is added, participation will be voluntary, anonymous and preceded by an information sheet and signed consent form (Appendix C), and no names, emails or raw prompts will be stored.

Evaluation data will be public benchmark prompts used under their licence terms. Prompts sent to hosted APIs will contain no personal data. Provider and client credentials will be protected as described in Section 3.6.7, and the Nigeria Data Protection Act 2023 will be followed for any personal data that is processed (Federal Republic of Nigeria, 2023). Use of generative AI tools in preparing this proposal is declared in the front matter, and all AI-assisted text, summaries and figures will be checked by the team against the original sources before submission.

3.11 Risk Assessment

Table 3.9: Risk assessment (L = likelihood, I = impact)

Risk	L	I	Mitigation
No benchmark has aligned scored outputs for the chosen models	M	H	Check coverage in Sprint 1; choose a smaller aligned pool from the same benchmark; report the change

Hosted API spending exceeds the cap	M	M	Use stored outputs for training and replay; hard spending limit on each provider key; small live pilot
Local model too slow or too large for laptop VRAM	M	M	Test quantised models in Sprint 1; use a smaller model; keep vLLM as an option
Hybrid predictor does not beat simple baselines	M	M	Treat as a valid finding; report the ablation honestly
Benchmark licence forbids the intended use	L	H	Check terms before download; switch to RouterBench or another licensed source
Provider model or price changes during the study	H	L	Record revisions and price tables per experiment; fix them for the test period
Scope creep (memory, online learning, extra models)	M	M	Freeze Must requirements after Sprint 2; log extras as future work
Team member unavailable during a sprint	M	M	Pair members on each module; shared repository and documentation

3.12 Work Plan

The work plan follows six two-week sprints across the two semesters, with report writing overlapping implementation. Owners follow the team roles in the statement of contributions.

Table 3.10: Sprint plan

Sprint	Weeks	Owners	Deliverable
1. Requirements, data access and environment	Sem. 1, Weeks 9–10	Wahida, Kabiru, Solomon	Requirements list with MoSCoW; benchmark coverage and licence check; environment on both laptops
2. Gateway, adapters and database	Sem. 1, Weeks 11–12	Waris, Maryam	OpenAI-compatible endpoint; Ollama and hosted adapters; schema and logging; frozen Must requirements
3. Baselines and replay harness	Sem. 1, Weeks 13–14	Maryam, Wahida, Solomon	Baseline policies; replay tool; pilot run to set the non-inferiority margin
4. Hybrid predictor and constrained policy	Sem. 2, Weeks 1–2	Solomon, Waris	Classifier, retrieval and hybrid estimates; constrained policy; validation tuning; targets frozen
5. Reliability controller and fault injection	Sem. 2, Weeks 3–4	Waris, Maryam	Breakers, bounded retries, response validation; mock provider; fault-injection results
6. Evaluation, dashboard and hardening	Sem. 2, Weeks 5–6	All; Kabiru (dashboard)	Test-set evaluation, ablations, live pilot; dashboard; release candidate

The exact calendar weeks will be aligned with the university timetable once it is published. Chapters Four and Five will be drafted during Sprints 5 and 6.

3.13 Budget

The team already owns the two laptops, and all software is free or open source, so the main costs are hosted API usage and incidental expenses. Amounts will be filled in from at least two written quotes or current provider price lists before submission.

Table 3.11: Budget

Item	Quantity	Cost (₦)	Source
Two RTX 4070 laptops (development and local model)	2	0 (owned)	Team members
Software (Python, FastAPI, PostgreSQL, Ollama, GitHub)	n/a	0 (free or open source)	Open source
Hosted API credit for live pilot and local-model checks	Capped pilot	[₦ amount from current price lists]	Team contribution
Internet data for downloads and API calls	6 sprints	[₦ amount]	Team contribution
Printing and binding of proposal and final report	2 submissions	[₦ amount from quotes]	Team contribution
Contingency (10%)	n/a	[₦ amount]	Team contribution
Total		[₦ total]	

3.14 Expected Outcomes

The project is expected to deliver a working routing gateway with source code; a requirements and design specification; a replay and fault-injection harness with an experiment protocol; an evaluation report comparing the hybrid router with baselines, including ablations and live pilot timing; and the final report. If the results are clear, the team will consider preparing a short paper.

3.15 Chapter Summary

The project will follow Design Science Research with Scrum development in six sprints. Requirements will come from existing gateways, provider documentation, the literature and failure catalogues, without human participants unless the supervisor asks for them. The design is a layered routing service with a constraint filter, a hybrid suitability predictor that combines a lightweight

classifier with embedding retrieval, a constrained selection policy and a reliability controller with circuit breakers and bounded fallback. It will be evaluated offline on licensed benchmark outputs with leakage-free splits, live on a capped pilot, and under injected failures, against simple baselines with paired statistical tests and ablations, with targets fixed before testing.

REFERENCES

Aderaldo, C. M., & Mendonça, N. C. (2026). ResilienceBench-Operator: A Kubernetes extension for systematic evaluation of microservice resilience patterns. *Journal of Software Engineering Research and Development, 14*, 179–208. https://doi.org/10.5753/jserd.2026.6965

Aggarwal, P., Madaan, A., Anand, A., Potharaju, S. P., Mishra, S., Zhou, P., Gupta, A., Rajagopal, D., Kappaganthu, K., Yang, Y., Upadhyay, S., Faruqui, M., & Mausam. (2024). AutoMix: Automatically mixing language models. In *Advances in Neural Information Processing Systems* (Vol. 37). Curran Associates. https://doi.org/10.52202/079017-4164

Agrawal, A., Agarwal, A., Kedia, N., Mohan, J., Kundu, S., Kwatra, N., Ramjee, R., & Tumanov, A. (2024a). *Etalon: Holistic performance evaluation framework for LLM inference systems* [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2407.07000

Chen, L., Zaharia, M., & Zou, J. (2024a). FrugalGPT: How to use large language models while reducing cost and improving performance. *Transactions on Machine Learning Research*. https://openreview.net/forum?id=cSimKw5p6R

Chen, S., Jia, Z., Khan, S., Krishnamurthy, A., & Gibbons, P. B. (2025). *SLOs-Serve: Optimized serving of multi-SLO LLMs* [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2504.08784

Chen, S., Jiang, W., Lin, B., Kwok, J. T., & Zhang, Y. (2024b). RouterDC: Query-based router by dual contrastive learning for assembling large language models. In *Advances in Neural Information Processing Systems* (Vol. 37). Curran Associates. https://doi.org/10.52202/079017-2120

Da, W., & Kalyvianaki, E. (2026). *RouteBalance: Fused model routing and load balancing for heterogeneous LLM serving* [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2606.17949

Dekoninck, J., Baader, M., & Vechev, M. (2025). A unified approach to routing and cascading for LLMs. In *Proceedings of the 42nd International Conference on Machine Learning* (Vol. 267, pp. 12987–13010). PMLR. https://proceedings.mlr.press/v267/dekoninck25a.html

Dholakia, K. (2026, July 13). *Auto Router v2: One router for complexity, semantic, and adaptive routing*. LiteLLM. Retrieved October 31, 2026, from https://docs.litellm.ai/blog/autorouter-v2

Ding, D., Mallick, A., Wang, C., Sim, R., Mukherjee, S., Rühle, V., Lakshmanan, L. V. S., & Awadallah, A. H. (2024). Hybrid LLM: Cost-efficient and quality-aware query routing. In *Proceedings of the Twelfth International Conference on Learning Representations*. https://openreview.net/forum?id=02f3mUtqnM

Ding, D., Mallick, A., Zhang, S., Wang, C., Madrigal, D., Garcia, M. D. C. H., Xia, M., Lakshmanan, L. V. S., Wu, Q., & Rühle, V. (2025). BEST-Route: Adaptive LLM routing with test-time optimal compute. In *Proceedings of the 42nd International Conference on Machine Learning* (Vol. 267, pp. 13870–13884). PMLR. https://proceedings.mlr.press/v267/ding25d.html

Federal Republic of Nigeria. (2023). *Nigeria Data Protection Act, 2023*. Nigeria Data Protection Commission. https://ndpc.gov.ng

Griggs, T., Liu, X., Yu, J., Kim, D., Chiang, W.-L., Cheung, A., & Stoica, I. (2024). *Mélange: Cost efficient large language model serving by exploiting GPU heterogeneity* [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2404.14527

Gujarati, A., Karimi, R., Alzayat, S., Hao, W., Kaufmann, A., Vigfusson, Y., & Mace, J. (2020). Serving DNNs like Clockwork: Performance predictability from the bottom up. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation* (pp. 443–462). USENIX Association. https://www.usenix.org/conference/osdi20/presentation/gujarati

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly, 28*(1), 75–105. https://doi.org/10.2307/25148625

Hu, Q. J., Bieker, J., Li, X., Jiang, N., Keigwin, B., Ranganath, G., Keutzer, K., & Upadhyay, S. K. (2024). *RouterBench: A benchmark for multi-LLM routing system* [Poster presentation]. Agentic Markets Workshop, International Conference on Machine Learning. https://doi.org/10.48550/arXiv.2403.12031

Huang, L., Magnusson, M., Muralikrishna, A. B., Estyak, S., Isaacs, R., Aghayev, A., Zhu, T., & Charapko, A. (2022). Metastable failures in the wild. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation* (pp. 73–90). USENIX Association. https://www.usenix.org/conference/osdi22/presentation/huang-lexiang

Huang, Z., Ling, G., Lin, Y., Chen, Y., Zhong, S., Wu, H., & Lin, L. (2025). RouterEval: A comprehensive benchmark for routing LLMs to explore model-level scaling up in LLMs. In *Findings of the Association for Computational Linguistics: EMNLP 2025* (pp. 3860–3887). Association for Computational Linguistics. https://doi.org/10.18653/v1/2025.findings-emnlp.208

Kitchenham, B., & Charters, S. (2007). *Guidelines for performing systematic literature reviews in software engineering* (EBSE Technical Report EBSE-2007-01). Keele University and Durham University.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J., Zhang, H., & Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles* (pp. 611–626). ACM. https://doi.org/10.1145/3600006.3613165

Li, H., Zhang, Y., Guo, Z., Wang, C., Tang, S., Zhang, Q., Chen, Y., Qi, B., Ye, P., Bai, L., Wang, Z., & Hu, S. (2026). LLMRouterBench: A massive benchmark and unified framework for LLM routing. In *Findings of the Association for Computational Linguistics: ACL 2026* (pp. 37733–37754). Association for Computational Linguistics. https://doi.org/10.18653/v1/2026.findings-acl.1881

LiteLLM. (n.d.-a). *Router: Load balancing*. Retrieved October 31, 2026, from https://docs.litellm.ai/docs/routing

LiteLLM. (n.d.-b). *[Beta] Auto routing*. Retrieved October 31, 2026, from https://docs.litellm.ai/docs/proxy/auto_routing

LiteLLM. (n.d.-d). *vLLM*. Retrieved October 31, 2026, from https://docs.litellm.ai/docs/providers/vllm

Liu, X., Chen, H., Lu, S., Ovadia, Y., Wen, G., Wu, H., Tan, Z., Zhang, J., Zedan, S., Kerido, Y., Weiss, L., Zhang, H., Yu, B., Balum, A., Limoy, N., Samara, A., Fan, B., Salisbury, B.,

Cook, R., . . . He, B. (2026). *vLLM Semantic Router: Signal driven decision routing for mixture-of-modality models* [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2603.04444

Lu, Y., Liu, R., Yuan, J., Cui, X., Zhang, S., Liu, H., & Xing, J. (2026). RouterArena: An open platform for comprehensive comparison of LLM routers. In *The Fourteenth International Conference on Learning Representations*. https://doi.org/10.48550/arXiv.2510.00202

Mohammadshahi, A., Shaikh, A. R., & Yazdani, M. (2024). *Routoo: Learning to route to large language models effectively* [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2401.13979

Moslem, Y., & Kelleher, J. D. (2026). Dynamic model routing and cascading for efficient LLM inference: A survey. *Transactions on Machine Learning Research*. https://openreview.net/forum?id=ypRg1TvQaM

Not Diamond. (n.d.-a). *What is model routing?* Retrieved October 31, 2026, from https://docs.notdiamond.ai/docs/what-is-model-routing

Not Diamond. (n.d.-c). *Training a custom router*. Retrieved October 31, 2026, from https://docs.notdiamond.ai/docs/router-training-quickstart

Not Diamond. (n.d.-e). *Pricing*. Retrieved October 31, 2026, from https://www.notdiamond.ai/pricing

Nygard, M. T. (2018). *Release it! Design and deploy production-ready software* (2nd ed.). Pragmatic Bookshelf.

Ong, I., Almahairi, A., Wu, V., Chiang, W.-L., Wu, T., Gonzalez, J. E., Kadous, M. W., & Stoica, I. (2025). RouteLLM: Learning to route LLMs with preference data. In *Proceedings of the Thirteenth International Conference on Learning Representations*. https://openreview.net/forum?id=8sSqNntaMr

OpenRouter. (n.d.-a). *Provider routing*. Retrieved October 31, 2026, from https://openrouter.ai/docs/guides/routing/provider-selection

OpenRouter. (n.d.-c). *Auto Router*. Retrieved October 31, 2026, from https://openrouter.ai/docs/guides/routing/routers/auto-router

Pandey, V., & Singh, G. (2026a). *ContinuityBench: A benchmark and systems study of stateful failover in multi-provider LLM routing* [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2607.15899

Pandey, V., & Singh, G. (2026b). *FailureAtlas: A taxonomy of failure modes in multi-provider LLM serving infrastructure* [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2607.17525

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Peffers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems, 24*(3), 45–77. https://doi.org/10.2753/MIS0742-1222240302

Portkey. (n.d.-a). *Load balancing*. Retrieved October 31, 2026, from https://portkey.ai/docs/product/ai-gateway/load-balancing

Portkey. (n.d.-b). *Fallbacks*. Retrieved October 31, 2026, from https://portkey.ai/docs/product/ai-gateway/fallbacks

Portkey. (n.d.-c). *Conditional routing*. Retrieved October 31, 2026, from https://portkey.ai/docs/product/ai-gateway/conditional-routing

Portkey. (n.d.-d). *Circuit breaker*. Retrieved October 31, 2026, from https://portkey.ai/docs/product/ai-gateway/circuit-breaker

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing* (pp. 3982–3992). Association for Computational Linguistics. https://doi.org/10.18653/v1/D19-1410

Romero, F., Li, Q., Yadwadkar, N. J., & Kozyrakis, C. (2021). INFaaS: Automated model-less inference serving. In *Proceedings of the 2021 USENIX Annual Technical Conference* (pp. 397–411). USENIX Association. https://www.usenix.org/conference/atc21/presentation/romero

Šakota, M., Peyrard, M., & West, R. (2024). Fly-swat or cannon? Cost-effective language model choice via meta-modeling. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining* (pp. 606–615). Association for Computing Machinery. https://doi.org/10.1145/3616855.3635825

Schwaber, K., & Sutherland, J. (2020). *The Scrum guide*. https://scrumguides.org/scrum-guide.html

Shirkavand, R., Gao, S., Yu, P., & Huang, H. (2025). Cost-aware contrastive routing for LLMs. In *Advances in Neural Information Processing Systems* (Vol. 38). https://proceedings.neurips.cc/paper_files/paper/2025/hash/e46eb6403af68506331f941282d838aa-Abstract-Conference.html

Shnitzer, T., Ou, A., Silva, M., Soule, K., Sun, Y., Solomon, J., Thompson, N., & Yurochkin, M. (2023). *Large language model routing with benchmark datasets* [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2309.15789

Tang, Y.-C., Hung, J.-C., Peng, W.-C., & Yen, A.-Z. (2026). *VDAR-Router: Adaptive LLMs routing via verbalized query difficulty analysis retrieval* [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2607.18098

Varangot-Reille, C., Bouvard, C., & Gourru, A. (2026a). Generalising LLM routing using past performance retrieval: A few-shot router is sufficient. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 4: Student Research Workshop)* (pp. 304–319). Association for Computational Linguistics. https://doi.org/10.18653/v1/2026.eacl-srw.22

Varangot-Reille, C., Bouvard, C., Ciancone, M., Gourru, A., Schaeffer, M., & Jacquenet, F. (2026b). Doing more with less: A survey on routing strategies for resource optimisation in large language model-based systems. *Journal of Artificial Intelligence Research, 86*. https://doi.org/10.1613/jair.1.19801

vLLM Semantic Router Team. (n.d.). *vLLM Semantic Router: An open, programmable decision layer for models and compute* [Computer software]. GitHub. Retrieved October 31, 2026, from https://github.com/vllm-project/semantic-router

Wohlin, C., Runeson, P., Höst, M., Ohlsson, M. C., Regnell, B., & Wesslén, A. (2012). *Experimentation in software engineering*. Springer. https://doi.org/10.1007/978-3-642-29044-2

Wu, F., & Silwal, S. (2025). Efficient training-free online routing for high-volume multi-LLM serving. In *Proceedings of the 39th Conference on Neural Information Processing Systems*. https://doi.org/10.48550/arXiv.2509.02718

Xiong, M., Hu, Z., Lu, X., Li, Y., Fu, J., He, J., & Hooi, B. (2024). Can LLMs express their uncertainty? An empirical evaluation of confidence elicitation in LLMs. In *The Twelfth International Conference on Learning Representations*. https://openreview.net/forum?id=gjeQKFxFpZ

Yan, H., Chen, Y., Ma, M., Wen, M., Lu, S., Zhang, S., Xu, T., Wang, R., Bansal, C., Rajmohan, S., Lin, Q., Zhang, C., & Zhang, D. (2025). *An empirical study of production incidents in generative AI cloud services* [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2504.08865

Zhang, Y., Li, H., Chen, J., Zhang, H., Ye, P., Bai, L., & Hu, S. (2025). Beyond GPT-5: Making LLMs cheaper and better via performance-efficiency optimized routing. In *Proceedings of the 2025 Seventh International Conference on Distributed Artificial Intelligence* (pp. 122–129). Association for Computing Machinery. https://doi.org/10.1145/3772429.3772445

Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., & Stoica, I. (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems* (Vol. 36). https://proceedings.neurips.cc/paper_files/paper/2023/hash/91f18a1287b398d378ef22505bf41832-Abstract-Datasets_and_Benchmarks.html

Zhong, Y., Liu, S., Chen, J., Hu, J., Zhu, Y., Liu, X., Jin, X., & Zhang, H. (2024). DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation* (pp. 193–210). USENIX Association. https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin

Zhu, Y., Liu, D., Lin, Z., Tong, W., Zhong, S., & Shao, J. (2025). The LLM already knows: Estimating LLM-perceived question difficulty via hidden representations. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing* (pp. 1160–1176). Association for Computational Linguistics. https://doi.org/10.18653/v1/2025.emnlp-main.61

Zhuang, R., Wu, T., Wen, Z., Li, A., Jiao, J., & Ramchandran, K. (2025). EmbedLLM: Learning compact representations of large language models. In *The Thirteenth International Conference on Learning Representations*. https://doi.org/10.48550/arXiv.2410.02223

APPENDICES

Appendix A: Source code repository link (to be added when the repository is created in Sprint 2)

Appendix B: Dataset and licence register for the benchmark sources

Appendix C: Requirements questionnaire, information sheet and consent form (only if the supervisor requires user input)

Appendix D: Test cases and traceability matrix (final report)

Appendix E: Generative AI use log

Appendix F: Project schedule (Gantt chart) and record of group meetings with the supervisor